

A Cross-Modal Prompt Injection Attack against Large Vision-Language Models with Image-Only Perturbation

Hao Yang
Xidian University
Xi'an, Shaanxi, China
yangh@stu.xidian.edu.cn

Zhuo Ma*
Xidian University
Xi'an, Shaanxi, China
mazhuo@mail.xidian.edu.cn

Yang Liu
Xidian University
Xi'an, Shaanxi, China
bcds2018@foxmail.com

Yilong Yang
Xidian University
Xi'an, Shaanxi, China
yilongyang@xidian.edu.cn

Guancheng Wang
Xidian University
Xi'an, Shaanxi, China
wangguancheng@stu.xidian.edu.cn

JianFeng Ma
Xidian University
Xi'an, Shaanxi, China
jfma@mail.xidian.edu.cn

Abstract

Large vision-language models (LVLMs) have emerged as a powerful paradigm for multimodal intelligence, but their growing deployment also expands the attack surface of prompt injection. Despite this growing concern, existing attacks still suffer from a critical limitation: the injected prompt for one modality only steers the model's interpretation of that singular input. Alternatively, these attacks remain *multimodal* but fail to achieve *cross-modal* prompt perturbation. To bridge this gap, we introduce a novel cross-modal prompt injection attack CROSSMPI, which can steer the model's interpretation of both textual and visual inputs via image-only perturbation. Our design is underpinned by the following key breakthroughs. First, we turn the focus of the injected prompt perturbation optimization from the visual embedding space (typically with only 10^5 parameters) to the model hidden state space (for multimodal information integration and with 10^7 parameters). Then, two strategies are adopted to mitigate the optimization challenges posed by the larger parameter space. To constrain the optimized model parameter space, we introduce a layer selection strategy that identifies the layers most critical to multimodal integration. Interestingly, deviating from the past experience, our analysis reveals that the optimal layers for LVLM prompt perturbation reside in the middle of the model rather than the last. To constrain the image perturbation space, we propose a new distance-decremental perturbation budget assignment strategy that allocates budgets decrementally as the pixel distance to semantic-critical regions increases. Extensive experiments across multiple LVLMs and datasets show that our method significantly outperforms baseline approaches.

CCS Concepts

• Security and privacy; • Computing methodologies → Artificial intelligence; Machine learning;

Keywords

Large Vision-Language Models; Prompt Injection Attacks; Cross-Modal Attacks

1 Introduction

Recently, Large Vision-Language Models (LVLMs) [1, 5, 7, 21, 53, 59, 68, 75] have rapidly advanced and emerged as a powerful paradigm

*Corresponding author.

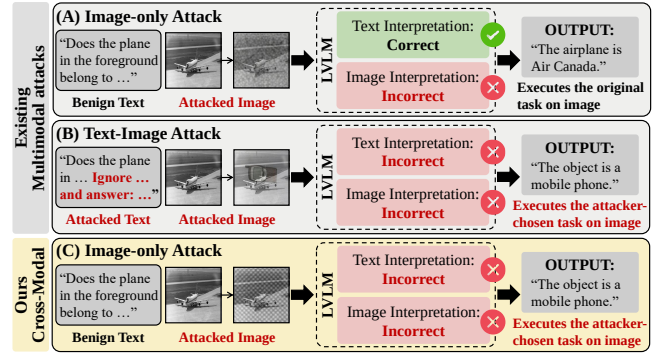


Figure 1: Comparison of prompt injection paradigms in LVLMs. (A) Image-only attacks perturb the image to change visual perception, while the model still follows the benign text task. (B) Text-image attacks change the task execution by perturbing both the text prompt and the image. (C) Our cross-modal prompt injection perturbs only the image but changes how the model interprets the unchanged text prompt.

for achieving unified multimodal intelligence. Unlike Large Language Models (LLMs) that focus solely on the text modality, LVLMs jointly model and reason over both visual and text inputs, enabling a wide range of complex cross-modal tasks, such as visual question-answering [2, 15, 49], text-to-image generation [29, 37, 40], and creative content creation [75]. By effectively integrating the complementary information encoded across different modalities, these models are capable of producing reasoning and decision-making that are more semantically coherent and context-aware, and they demonstrate substantial potential for continued expansion in the pursuit of general artificial intelligence and real-world applications.

Alongside their growing capabilities, recent studies have shown that LVLMs inherit and amplify security vulnerabilities originally observed in LLMs, among which *prompt injection attacks* pose a particularly severe threat. Prompt injection [23, 32, 43, 48] introduces malicious instructions that override or conflict with the user's original intent, causing the model to execute an attacker-chosen injected task rather than the intended target task. In LLM-based systems, attackers typically achieve this by inserting carefully crafted strings into user instructions (e.g., "ignore all previous instructions

and instead ...) to override the original instruction logic [33]. In multimodal settings, however, this threat becomes more stealthy and powerful: malicious instructions can be embedded not only explicitly within text prompts but also implicitly encoded in visual inputs [16, 51, 58]. Such prompt injection attacks can steer models toward entirely different tasks [23] or exert fine-grained control over model outputs [34], and have been dubbed the #1 security risk for applications integrating LLMs by OWASP [31].

Despite this growing concern, existing prompt injection attacks against LVLMs still share a critical limitation: injecting prompts into one modality only serves to steer the model's interpretation of that singular input. With more details, as shown in Figure 1, current methods mainly fall into two categories. The first is prompt injection with only image perturbed [16, 58]. In this scenario, the model still follows the task defined by the benign text prompt, but its perception of the image is perturbed. The second is prompt injection with both text and image prompts perturbed [51]. Here, although the model's perception of both text and image prompts is manipulated, the misperception in the textual domain is not caused by the image prompt perturbation but the text.

In this paper, we propose a cross-modal prompt injection attack CROSSMPI, in which the attacker can use image-only perturbation to change the model's interpretation of both the visual and textual prompts. The key insight behind CROSSMPI is that, for an LVLm, its visual evidence and textual intent integration occurs in its hidden state space but not the embedding space. As a result, unlike the prior works [51, 58], all of which optimize its prompt perturbation over the embedding space, CROSSMPI turns its target to the model hidden state space.

The primary challenge brought by the target change is the explosively expanded parameter space for perturbation optimization. For example, in a 7B-MiniGPT4 model, the visual embedding space contains only about 10^5 parameters, whereas the hidden state space can reach about 10^7 parameters. Even worse, the hardness is further amplified by the size of image prompts. That means, with a m -dimensional hidden state space and an image prompt with size n , the perturbation optimization complexity is at least $O(mn)$. Naturally, directly optimizing over the full hidden state space is computationally expensive and can easily overfit to model-specific activations, leading to low transferability of the generated perturbations. To address this problem, CROSSMPI introduces two strategies: a fusion-critical layer selection strategy for the constraint of m , and a distance-decremental perturbation budget assignment strategy for the constraint of n .

For fusion-critical layer selection, we first compare attack performance when perturbations are optimized on the early, middle, and final layers of LVLms. Interestingly, deviated from the past experience that the final layers contribute most to adversarial perturbation optimization [6, 26, 67], we find that the most effective layers for cross-modal prompt perturbation are located in the middle of LVLms. This is because middle layers are where textual intent and visual evidence are actively fused into task-level representations, whereas the final layers mainly map these already formed latent decisions to the output tokens [28, 65, 72].

Based on this insight, CROSSMPI focuses on the middle layers and further proposes an algorithm to identify which of them are

most responsible for multimodal fusion. Specifically, CROSSMPI uses layer-wise probing to measure how the hidden states at each layer react to different text prompt changes. Layers that remain stable under wording or syntax changes but show clear changes when the task semantics change are treated as fusion-critical layers. These layers capture the stage where the model binds visual evidence with the textual instruction. Restricting optimization to them reduces the number of optimized hidden states and keeps the attack focused on the representations that determine task interpretation.

For perturbation budget assignment, we construct a distance-decremental budget masking mechanism to concentrate the image perturbation on semantic-critical regions. The most straightforward way to redistribute the budget is to use saliency scores, assigning larger budgets to pixels that contribute more to the model prediction [71]. However, gradient-based saliency may also highlight background or task-irrelevant regions that are correlated with the model prediction, even though these regions are not the main visual evidence. Allocating large budgets to such scattered regions weakens the perturbation on truly important areas and reduces attack efficiency. To address this, we introduce a spatial distance penalty centered on the semantic-critical region. The perturbation budget is high only for pixels that are salient and close to this center, and it gradually decreases as pixels move farther away.

Extensive experiments demonstrate the effectiveness of our attack across diverse datasets and LVLms. Specifically, in the black-box setting, CROSSMPI achieves a 66.36% success rate in inducing models to execute the attacker-chosen task, outperforming four state-of-the-art baselines by 40.91 percentage points on average. Both qualitative visualizations and quantitative image similarity metrics further show that CROSSMPI preserves excellent imperceptibility. In addition, ablation studies validate the contribution of each key component and demonstrate the robustness of CROSSMPI under different attack scenarios.

Our contributions can be summarized as follows:

- We propose CROSSMPI, a cross-modal prompt injection attack against LVLms, which can utilize image-only perturbation to manipulate the interpretation of LVLms on both the visual and textual prompts.
- We explore a new direction to achieve perturbation optimization for prompt injection, i.e., optimizing over the model hidden state space instead of the embedding space.
- We design two strategies to constrain the optimization space for cross-modal prompt injection: 1) the fusion-critical layer selection strategy and 2) the distance-decremental perturbation budget assignment strategy. During the design of the first strategy, we find that the most critical layers for cross-modal prompt perturbation optimization are located in the middle of LVLm but not the final layers, which is deviated from the past experience for adversarial perturbation optimization.
- Extensive evaluations across LVLms and datasets demonstrate that, in the black-box setting, CROSSMPI achieves a 66.36% success rate in inducing models to execute the attacker-chosen task, outperforming baselines by 40.91 percentage points on average.

2 Preliminaries

2.1 Large Vision-Language Models

Large Vision-Language Models (LVLMs) extend large language models with visual perception so that a single model can process a text-image pair and generate task-dependent language outputs, enabling applications such as image captioning, visual question answering, and vision-language dialogue. A typical LVLM consists of a vision encoder, a learnable projection module, and an autoregressive language model. Formally, let x_v denote an input image and x_p denote a text prompt. The image is first processed by a vision encoder $f_{vis}(\cdot)$ to produce a sequence of visual tokens $t_v = \{t_v^1, t_v^2, \dots, t_v^{T_v}\}$, where $t_v^i \in \mathbb{R}^{d_v}$, T_v is the number of visual tokens and d_v is the feature dimension of the vision encoder. Because the vision encoder and the language model generally operate in different embedding spaces, a projection module $f_w(\cdot)$ maps the visual tokens into the hidden space of the language model, yielding $e_v = f_w(t_v)$ with $e_v \in \mathbb{R}^{T_v \times d_l}$, where d_l is the hidden dimension of the language model. In parallel, the prompt x_p is tokenized and embedded into text tokens $e_p = \{e_p^1, e_p^2, \dots, e_p^{T_p}\}$, where $e_p^i \in \mathbb{R}^{d_l}$ and T_p is the number of textual tokens. The projected visual tokens and text tokens are concatenated into a joint multimodal sequence $e = [e_v; e_p]$, which is then processed by the language model $f_\phi(\cdot)$ to autoregressively generate an output sequence $y = (y_1, y_2, \dots, y_{T_y})$.

From a probabilistic perspective, an LVLM models the conditional distribution of the output text given both visual and textual inputs, denoted by $P_\theta(y | x_p, x_v)$, where θ collects the parameters involved in multimodal inference. The response distribution follows the standard autoregressive factorization

$$P_\theta(y | x_p, x_v) = \prod_{i=1}^{T_y} P_\theta(y_i | y_{<i}, x_p, x_v), \quad (1)$$

where $y_{<i} = \{y_1, y_2, \dots, y_{i-1}\}$. At each generation step, the model produces a logit vector $z_i \in \mathbb{R}^{|V|}$ over the vocabulary V , which is transformed into a probability distribution over tokens by a softmax layer before decoding.

Training and Inference. Given a training dataset D , training is typically formulated as minimizing the negative log-likelihood of the ground-truth outputs:

$$\mathcal{L}(\theta) = \mathbb{E}_{(x_p, x_v, y) \sim D} [-\log P_\theta(y | x_p, x_v)]. \quad (2)$$

This objective encourages the model to bind visual evidence and textual instructions into a coherent multimodal representation that supports downstream generation. At inference time, the parameters θ are fixed, and the model generates responses autoregressively from the learned conditional distribution, typically via greedy decoding or sampling-based strategies with temperature control.

2.2 Prompt Injection Attack

Prompt injection attacks [23, 32, 43] manipulate model behavior by injecting attacker-controlled content into the input without changing the model parameters. These attacks exploit the difficulty of instruction-following models in reliably separating trusted instructions from untrusted user-supplied content, allowing attackers to override the intended task, redirect model behavior, or induce unintended outputs. For LVLMs, the attack surface is broader because

malicious signals can be introduced through either the textual or visual modality [16, 58].

Formally, given a trained LVLM $\mathcal{M}$ with fixed parameters θ , a user provides a text-image input pair (x_p, x_v) . The text prompt x_p defines the user-intended task $\mathcal{T}$, denoted as $\mathcal{T} \leftarrow x_p$, and the model output is

$$y = \mathcal{M}(x_p, x_v). \quad (3)$$

In a multimodal prompt injection attack, the attacker aims to redirect the model from the user-intended task $\mathcal{T}$ to an attacker-chosen target task $\mathcal{T}_t$. We characterize the injected task using a target text-image pair (x_p^t, x_v^t) , where $\mathcal{T}_t \leftarrow x_p^t$, and the desired target model output is

$$y_t = \mathcal{M}(x_p^t, x_v^t). \quad (4)$$

Instead of directly replacing the benign input with (x_p^t, x_v^t) , the attacker can inject perturbations into both modalities by constructing a text perturbation Δ_p and an image perturbation Δ_v . The attack objective is to make the modified input $(x_p + \Delta_p, x_v + \Delta_v)$ induce the attacker-chosen task and produce the target output:

$$\mathcal{M}(x_p + \Delta_p, x_v + \Delta_v) = \mathcal{M}(x_p^t, x_v^t) = y_t, \quad (5)$$

where the injected text perturbation changes the task from $\mathcal{T}$ to $\mathcal{T}_t$, and the image perturbation provides additional visual evidence aligned with the target output. In this setting, the malicious instruction can be carried through both the textual and visual modalities, causing the LVLM to follow the attacker-specified task rather than the user's original task. Different from prior multimodal attacks that modify both the text and image inputs, our attack uses image-only perturbations to change the model's interpretation of the text-image input and induce it to execute the attacker-chosen task. Formally, we define the two attack settings as follows:

DEFINITION 1 (MULTIMODAL VS. CROSS-MODAL PROMPT INJECTION ATTACKS). Assume an LVLM $\mathcal{M}(\cdot, \cdot)$ and a text-image input pair (x_p, x_v) . Let $\mathcal{T} \leftarrow x_p$ denote the intended task defined by x_p , and $y = \mathcal{M}(x_p, x_v)$ be the output of $\mathcal{M}$.

- **Multimodal Prompt Injection:** Find perturbations Δ_p and Δ_v to make $\mathcal{T}_t \leftarrow x_p + \Delta_p$ and $y_t = \mathcal{M}(x_p + \Delta_p, x_v + \Delta_v)$, where y_t and $\mathcal{T}_t$ are attacker-chosen but $\mathcal{T}_t \neq \mathcal{T}$ holds if and only if $\Delta_p \neq 0$.
- **Cross-Modal Prompt Injection:** Find a perturbation Δ_v to make $\mathcal{T}_t \leftarrow x_p$ and $y_t = \mathcal{M}(x_p, x_v + \Delta_v)$ where both $\mathcal{T}_t \neq \mathcal{T}$ and $y_t \neq y$ are attacker-chosen.

3 Threat Model

We consider CROSSMPI under a standard LVLM application scenario. Our threat model is defined along two dimensions: the attacker's goal and the attacker's capabilities.

Attacker's Goal. The attacker aims to change the task executed by the LVLM by modifying only the image input. Under this setting, the attacked image should cause the LVLM to execute an attacker-chosen injected task instead of the user-intended task. Given a benign text-image pair (x_p, x_v) , where x_p is the text prompt and x_v is the original image, the attacker generates a perturbed image $x_v' = x_v + \Delta_v$. The text prompt remains unchanged. The goal is to make the model interpret (x_p, x_v') as an attacker-chosen injected task and produce the attacker-chosen output y_t . Formally, the attack seeks to satisfy

$$\mathcal{M}(x_p, x_v + \Delta_v) = y_t, \quad (6)$$

while keeping Δ_v visually imperceptible. Thus, the attack is not simply to cause an incorrect answer, but to redirect the model’s task understanding through the visual modality alone.

Attacker’s Capabilities. We consider a black-box target setting where the attacker cannot access the deployed target LVLM, including its parameters, gradients, intermediate hidden states, system prompt, safety policy, or inference configuration. The attacker can freely choose the injected task, including the target text prompt and target image, and thereby define the desired target output. The attacker also knows the benign input pair, including the text prompt x_p and the original image x_v . To construct the attack, the attacker can use publicly available LVLMs offline to optimize a bounded image perturbation Δ_v , producing the attacked image $x'_v = x_v + \Delta_v$. At deployment time, the attacker can only submit the attacked image x'_v with the unchanged benign text prompt to the target LVLM. These adversarial examples can subsequently compromise the functionality of target systems, such as misleading VLM-based web agents [61] or disrupting real-world object detectors [45].

4 Methodology

In this section, we present the methodology of CrossMPI under our threat model. As shown in Figure 3, CrossMPI consists of three main stages. First, fusion-critical layer selection reduces the parameter search space by limiting optimization to a small number of layers most relevant to multimodal fusion. Second, distance-decremental perturbation budget assignment reduces the image search space by concentrating the perturbation budget around semantic-critical regions and decreasing it with distance. Third, cross-modal perturbation optimization learns the final image perturbation using output-level supervision, fusion-layer hidden states alignment, and frequency regularization.

4.1 Fusion-Critical Layer Selection

To constrain the optimized model parameter space, we first analyze the attack performance obtained from different LVLM layer groups. Specifically, we consider early layers, middle layers, final layers, and their pairwise combinations, resulting in six optimization settings. For each setting, we apply the optimization objective introduced in Section 4.3 to the selected layers. The detailed setup and results are reported in Section 5.3.1. Unlike prior methods that mainly optimize final-layer representations [6, 26, 67], our results show that optimizing middle-layer representations achieves the best attack performance. This indicates that optimizing cross-modal representations at the fusion stage can more effectively steer the model’s task interpretation. Based on this observation, we focus on the middle layers and further introduce a fusion-critical layer selection strategy to identify the layers that contribute most to vision-text fusion.

Notably, existing approaches for studying LVLM internals mainly fall into three categories: 1) Attention knockout [28], masks or removes specific attention connections and quantifies the importance of layers or attention heads according to the resulting degradation in model predictions; 2) Logit lens [30, 72], maps hidden states of each layer to the vocabulary space through the language-model head and examines how output-related information emerges across the network; 3) Layer-wise probing [65], trains classifiers on the

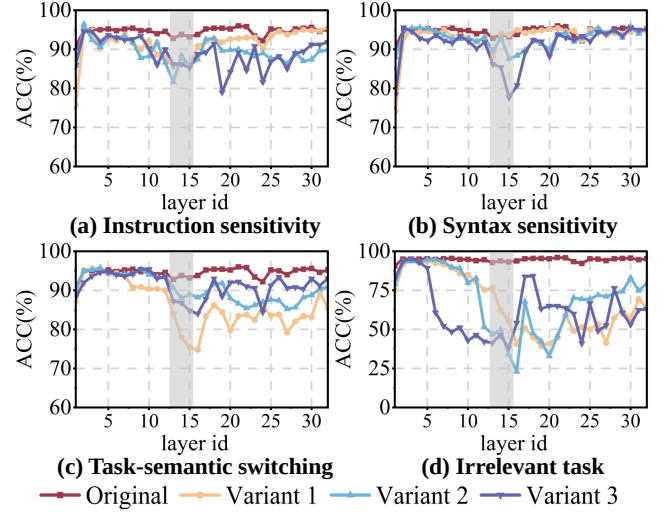


Figure 2: Layer-wise hidden state classification accuracy under prompt variants on MiniGPT4-vicuna. The fusion-critical layers are identified as layers 13–15.

hidden states of each layer and measures how sensitive these representations are to different prompt variations. Among them, attention knockout focuses on information-flow paths, while logit lens measures the relationship between intermediate hidden states and output distributions. In contrast, layer-wise probing directly evaluates the semantic information encoded in each layer’s hidden states and its stability under prompt changes. Therefore, we adopt this method to identify fusion-critical layers, defined as layers whose probing accuracy changes substantially when the prompt semantics or task is modified. Our method consists of two main steps:

Step-1: Layer-wise classifier training. We first feed the LVLM with animal images paired with the benign prompt “What is the animal in the picture?”. For each transformer layer l , we extract the hidden state of the last input token, which summarizes the preceding visual and textual tokens under causal attention, and therefore provides a compact representation of the prompt-conditioned multimodal context. We then use the hidden states from each layer to train an independent Multilayer Perceptron (MLP) classifier $g^{(l)}$ for predicting the visual category. Once trained, all classifiers are frozen and used as a fixed evaluator for evaluating hidden states produced under different prompt variants.

Step-2: Text-variant sensitivity testing. Prior method [65] only uses lexical variants (e.g., replacing “picture” with “image”) and semantic negation variants (e.g., replacing “animal” with “plant”). However, both variants are constructed through a single-word substitution, making it difficult to distinguish whether changes in hidden states come from local word replacement or from the resulting semantic shift. Therefore, we introduce four complementary prompt variants for fusion-layer selection. *Instruction Sensitivity* changes the instruction wording while keeping the queried content, such as changing “What is” to “Tell me”, to test whether a layer is affected by superficial instruction wording. *Syntax Sensitivity* rewrites the sentence structure without changing the task meaning,

such as using an inverted sentence form, to test sensitivity to syntactic structure. *Task-Semantic Switching* changes the queried concept, such as changing “animal” to “plant”, to test whether textual semantics begin to reshape the encoded visual evidence. *Irrelevant-Task Replacement* replaces the original query with an unrelated task, such as “What color is the sky?”, to test whether a layer encodes task-level intent rather than local lexical cues. We provide three prompts for each variant in Table 6. We then feed each image with these prompt variants into the LVLM, extract the hidden states from each layer, and evaluate them using the corresponding frozen MLP classifier.

Figure 2 shows the layer-wise probing results of MiniGPT4-vicuna [75]. In the early layers, classification accuracy remains stable under instruction, syntax, and task-semantic changes, but drops sharply under irrelevant-task replacement, indicating that these layers mainly capture broad task context. In the middle layers, different prompt variants lead to clear differences: instruction and syntax changes cause only small accuracy drops, whereas task-semantic switching and irrelevant-task replacement cause much larger drops. This indicates that textual semantics begin to reshape visual representations in this depth range. For example, under the first task-semantic switching variant, the accuracy at layers 13–15 decreases by about 25%. After these layers, the accuracy curves become stable, suggesting that multimodal inference has largely been completed. We therefore identify layers 13–15 as the fusion-critical region of MiniGPT4-vicuna. More experimental details and results are provided in Appendix C.

4.2 Distance-Decremental Perturbation Budget Assignment

To constrain the image optimization space, we design a distance-decremental perturbation budget mask. The goal is to assign larger perturbation budgets to regions that are both semantically important and spatially close to the main evidence region. A straightforward way to allocate the budget is to rely only on saliency scores, giving larger budgets to pixels with higher saliency [71]. However, gradient-based saliency can be scattered over background regions that are correlated with the prediction but do not provide the core visual evidence. Moreover, LVLMs can often recognize an object from only a few discriminative pixels [63, 71]. Therefore, the perturbation should cover the target object as completely as possible, rather than being concentrated only on a few highly salient pixels. To address this problem, we introduce the distance to the semantic-critical center as a spatial penalty, making the perturbation budget decrease as pixels move farther from this center. This design keeps the perturbation concentrated around the most relevant semantic region while still allowing nearby contextual pixels to contribute. Our method consists of two main steps:

Step-1: Saliency calculation. Different LVLMs may focus on different image regions for the same input. To obtain a more model-independent saliency estimate, we first use a general-purpose reference model, e.g., GPT-5.4-mini, to generate a concise image description. Specifically, we query the model with the fixed prompt “Describe the image in one sentence.” and obtain a summary y . This summary captures the main visual semantics of the image and serves as a text condition for saliency estimation. By aligning the

image with this neutral description, we can locate the regions that are most relevant to the image semantics while reducing dependence on the architectural bias of the attacked LVLM.

Specifically, we utilize Grad-ECLIP [71, 73] to compute a saliency map on the description. Grad-ECLIP measures image-text relevance by backpropagating the similarity signal between the visual and text representations to the visual tokens, and then projects the token-level relevance scores back to the image space. Given the image x_o and its description y , this process can yield an importance map S , where s_{ij} denotes the saliency score of pixel (i, j) .

Step-2: Distance-decremental perturbation budget mask construction. Given the saliency map S , we first normalize the importance score of each pixel:

$$r_{ij} = \frac{s_{ij}}{\max_{i,j}(s_{ij})}, \quad (7)$$

where $r_{ij} \in [0, 1]$ denotes the relative semantic importance of pixel (i, j) . We then estimate the semantic-critical center of the image from the most salient pixels. Specifically, we select the top- $k\%$ pixels with the highest normalized saliency scores as the semantic-critical support U , and compute their weighted centroid:

$$c = \frac{\sum_{(i,j) \in U} r_{ij} \cdot (i, j)}{\sum_{(i,j) \in U} r_{ij}}. \quad (8)$$

This center represents the main region that contributes to the image-level semantics. Next, we compute a distance penalty for each pixel based on its distance to the semantic-critical center. For each pixel (i, j) , we calculate its Euclidean distance to c , denoted by $v_{ij} = \|(i, j) - c\|_2$, and normalize it as

$$d_{ij} = \frac{v_{ij}}{\max_{i,j}(v_{ij})}. \quad (9)$$

We combine semantic importance and spatial distance to obtain the budget weight:

$$w_{ij} = 1 + r_{ij} - (1 - r_{ij}) \cdot d_{ij}. \quad (10)$$

Here, r_{ij} increases the weight of semantically important pixels, while $(1 - r_{ij}) \cdot d_{ij}$ penalizes pixels that are both less salient and farther from the semantic-critical center. Therefore, important regions are retained, whereas distant regions with weak semantic relevance are downweighted. Finally, we convert the weight map into a local perturbation budget mask:

$$\epsilon_{ij} = \epsilon \left(1 - \lambda + \lambda \cdot \frac{w_{ij}}{\bar{w}} \right), \quad (11)$$

where $\bar{w}$ is the average weight, ϵ denotes the base perturbation budget, and $\lambda \in [0, 1]$ controls the strength of budget redistribution. Normalizing by $\bar{w}$ keeps the average perturbation budget across the image unchanged, so the mask redistributes the total budget rather than increasing it. When $\lambda = 0$, all pixels share the same budget. As λ increases, pixels with larger weights receive more budget, while distant and weakly relevant pixels receive less. As a result, the final mask concentrates perturbations around semantic-critical regions and gradually reduces the budget for less relevant areas.

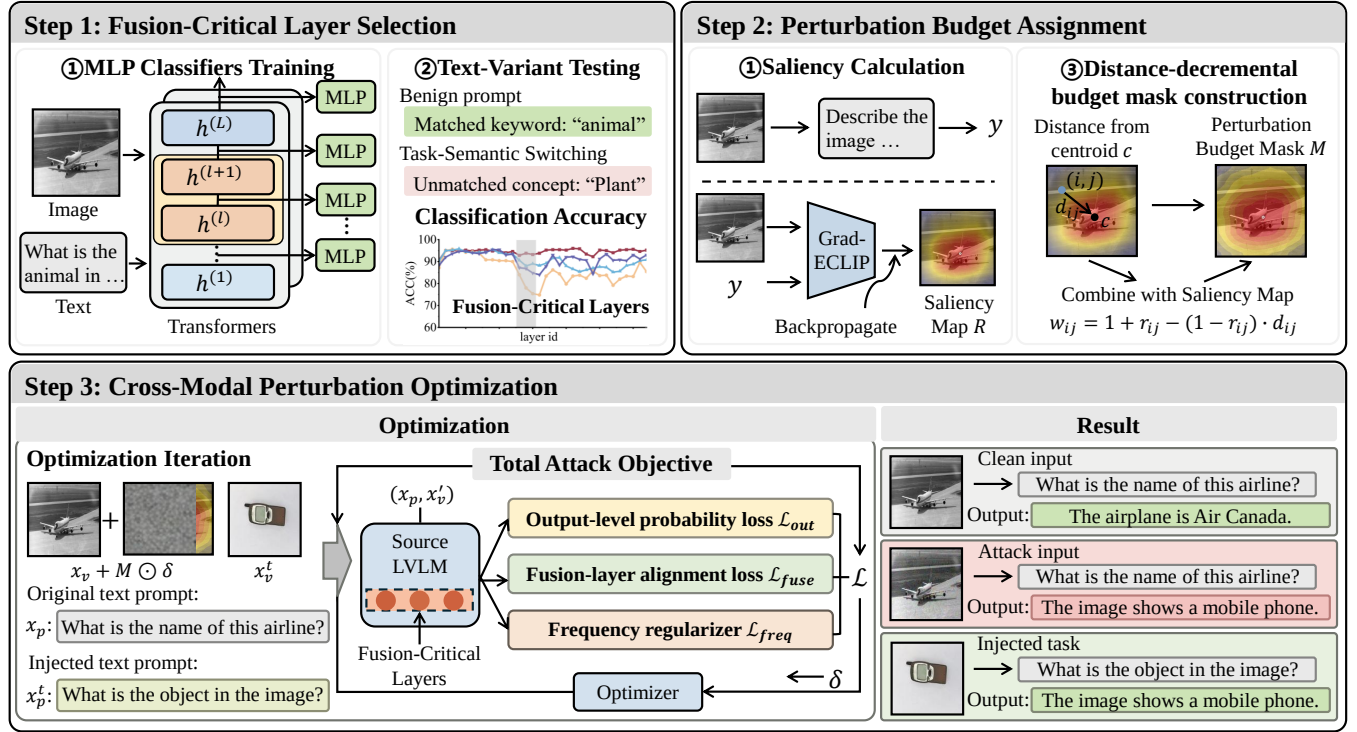


Figure 3: Overview of our attack. CrossMPI consists of three parts: (1) fusion-critical layer selection, which localizes the layers where visual evidence and textual intent are integrated; (2) perturbation budget assignment, which uses Grad-ECLIP saliency and distance-decremental weighting to construct a perturbation budget mask for budget reallocation; (3) Cross-Modal Perturbation Optimization, which jointly optimizes output-level, fusion-level, and frequency-domain objectives.

4.3 Cross-Modal Perturbation Optimization

Prior image-based prompt injection methods mainly optimize perturbations in the visual embedding space. Although such perturbations can change how the image is represented, they fail to fundamentally change how the model interprets the textual instruction. The reason is that the injected noise primarily affects visual features before multimodal fusion, while the task semantics are determined later through the interaction between visual evidence and the text prompt. To address this limitation, we turn the optimization target from the visual embedding space to the model hidden state space, where visual and textual information are jointly integrated. By integrating the reduction strategies detailed in Sections 4.1 and 4.2, we significantly prune both the parameter and image optimization search space. Specifically, letting M denote the perturbation budget mask whose entries are given by the local thresholds ϵ_{ij} and δ the learnable perturbation, we define the effective perturbation as $\Delta_v = M \odot \delta$ and construct the attacked image as

$$x_v' = x_v + \Delta_v = x_v + M \odot \delta. \quad (12)$$

The mask controls where the perturbation is allowed to be stronger, so that the optimization focuses on image regions that are more likely to affect multimodal task grounding.

To enable the perturbation to successfully alter the model’s interpretation of the text prompt and perform an attacker-chosen

task, we propose a dual-faceted strategy that optimizes the perturbation at two complementary levels. The first level is output-level optimization, which directly increases the probability of the target task response. Its role is to ensure that the perturbed image can successfully modify the final model output. The second level is fusion-level optimization, which aligns the hidden states of the selected fusion-critical layers with the target task representation. Its role is to intervene in the intermediate representation space, ensuring that the perturbation changes how the model interprets the benign text prompt and thereby improving transferability.

The first optimization objective is output-level optimization, which makes the attacked image directly induce the attacker-chosen response under the benign text prompt. Specifically, given the attacker-selected target task input (x_p^t, x_v^t) and keywords of target output token sequence $y_t = \{y_1^t, \dots, y_T^t\}$. After applying the perturbation, the attacked image is given by $x_v' = x_v + M \odot \delta$. We then optimize the perturbation by maximizing the likelihood of the target output conditioned on the benign prompt and the attacked image. The output-level loss is defined as

$$\mathcal{L}_{out} = - \sum_{i=1}^T \log P_{\mathcal{M}}(y_i^t | y_{<i}^t, x_p, x_v'), \quad (13)$$

where $\mathcal{M}$ denotes the source LVLM. This objective explicitly forces the attacked input to place high probability mass on the target task token sequence, thereby aligning the optimization with the

actual malicious goal. However, as the final response is produced after multiple stages of multimodal fusion, language decoding, and model-specific alignment, the optimized perturbations can only capture source-specific features, which perform well in white-box evaluation but transfer poorly to other models.

The second component is fusion-level optimization, which makes the benign prompt and the perturbed image form an internal representation similar to that of the attacker-chosen task. Let $\mathcal{L}_{sel}$ denote the selected fusion-critical layers, and let $H^{(l)}(x_p, x_v)$ be the hidden state at layer l . We then minimize the distance between their hidden states at the selected fusion-critical layers:

$$\mathcal{L}_{fuse} = \sum_{l \in \mathcal{L}_{sel}} \left\| H^{(l)}(x_p, x'_v) - H^{(l)}(x'_p, x'_v) \right\|_2^2. \quad (14)$$

This loss directly constrains the intermediate fusion process: it pulls the hidden states of the attacked input toward the hidden states of the target task. As a result, the perturbation is encouraged to change how the model interprets the benign text prompt, rather than only changing the final decoded answer. By supervising the selected fusion-critical layers, this objective provides more stable and semantically meaningful gradients, which improve optimization stability and cross-model transferability.

To improve visual stealthiness and reduce overfitting to the source model, we introduce a frequency regularization term to suppress high-frequency components in the perturbation. Specifically, we transform the perturbation Δ_v into the frequency domain, remove the central low-frequency region, and penalize the mean magnitude of the remaining high-frequency components:

$$\mathcal{L}_{freq} = \frac{1}{|\Omega_h|} \sum_{(u,v) \in \Omega_h} |\mathcal{F}(\Delta_v)_{u,v}|, \quad (15)$$

where $\mathcal{F}(\cdot)$ denotes the two-dimensional Fourier transform and Ω_h denotes the high-frequency region outside the central low-frequency window. This loss penalizes high-frequency noise, which can create visible artifacts and overfit to the preprocessing or patch-embedding behavior of the source model. In this way, $\mathcal{L}_{freq}$ encourages smoother perturbations that better preserve visual appearance and transfer robustly across models.

Finally, to improve robustness to common image variations, we optimize the perturbation over multiple transformed views instead of a single fixed image. Specifically, for each optimization step, we construct a transformation augmentation set $\mathcal{A} = \{\tau_0, \tau_1, \dots, \tau_5\}$, where τ_0 is the identity mapping and the remaining five transformations correspond to scaling, rotation, brightness adjustment, blur, and additive noise. The attack objective is then enforced on all six views $\{\tau(x'_v) \mid \tau \in \mathcal{A}\}$, so that the learned perturbation remains effective under benign distortions introduced during image transmission, preprocessing, or acquisition, yielding a more stable attack in practical settings. We combine the three terms into the final attack objective:

$$\mathcal{L}_{attack} = \frac{1}{|\mathcal{A}|} \sum_{\tau \in \mathcal{A}} [\mathcal{L}_{out}(\tau(x'_v)) + \alpha \mathcal{L}_{fuse}(\tau(x'_v)) + \beta \mathcal{L}_{freq}(\tau(\Delta_v))], \quad (16)$$

where α and β control the weights of the fusion-layer alignment loss and the frequency regularization loss, respectively. Taking into account these three complementary optimization goals, we can

make the perturbation affect both the model's interpretation of the benign prompt and its final response, while improving stealth and transferability.

5 Experimental Evaluation

In this section, we conduct comprehensive evaluations for CrossMPI. We first introduce our experimental setup and then present the detailed results of each evaluation.

5.1 Experimental Setup

Datasets. We evaluate CrossMPI on three datasets: MSCOCO [19], ImageNet [41], and TextVQA [44]. We use VQA questions [4, 25] as text prompts to define distinct benign and attacker-chosen tasks. For MSCOCO and ImageNet, the attacker-chosen target task is "What is the animal in the picture?", with sheep and cat as the target images, respectively. For TextVQA, the target task is "What is the object in the picture?", with a phone as the target image. We randomly sample 500 examples from each dataset for evaluation, while ensuring that the sampled images do not belong to the same category as the corresponding target image. Detailed dataset construction and preprocessing settings are provided in Appendix D.

Models. We evaluate six representative LVLs, namely MiniGPT4-llama2 [75], MiniGPT4-vicuna [75], InstructBLIP [7], BLIP-2 [15], BLIVA [12], and Qwen2.5-VL [47]. All models are used with their original released settings, without architecture modifications or customized inference configurations. We use MiniGPT4-llama2, MiniGPT4-vicuna, InstructBLIP, and Qwen2-VL [52] as source models for perturbation optimization. The optimized adversarial examples are then evaluated on the target LVLs to measure both source model attack effectiveness and cross-model transferability. Unless otherwise specified, we optimize three fusion-critical layers and set the perturbation budget to 16/255.

Evaluation metrics. We use two metrics to evaluate attack performance: Attack Success Rate (ASR) [14, 43, 71] and Semantic Similarity (SS) [4, 20, 62]. ASR measures the proportion of test samples for which the perturbed image makes the model follow the attacker-chosen task, and therefore reflects the effectiveness of the attack. SS measures the semantic similarity between the model response and the attacker-specified target text. Specifically, we encode both the attacked response and the target text using Sentence-Transformer [38] with the all-MiniLM-L6-v2 encoder, and compute their cosine similarity as SS. This metric evaluates whether the attack induces the intended malicious semantics rather than arbitrary output changes. Reporting both metrics is necessary because a successful cross-modal prompt injection attack should achieve both high attack success and high semantic alignment.

Baselines. We compare CrossMPI with four representative attacks: ARE-W [58], ARE-B [58], CI [51], and ATPI [16]. ARE-W is a white-box attack that directly optimizes the perturbed image to increase the probability of the attacker-chosen output. ARE-B is a black-box variant that uses CLIP as a surrogate model to increase the similarity between the perturbed image embeddings and the target output embeddings. CI also uses CLIP-based optimization to increase the similarity between the perturbed image embeddings and target image embeddings, while additionally optimizing the text prompt to increase the probability of the target output. In our

Table 1: Overall attack performance across MSCOCO, ImageNet, and TextVQA. ASR (%) and SS (%) compare baseline attacks with our method under the best source model for each target LVLm. The best results are highlighted in bold font.

Datasets	Target Models	Best Source Model	ARE-W		ARE-B		CI		ATPI		Ours	
			ASR	SS	ASR	SS	ASR	SS	ASR	SS	ASR	SS
MSCOCO	MiniGPT4-llama2	MiniGPT4-vicuna	1.35	37.82	15.38	40.10	19.05	30.94	0.00	29.05	28.43	43.17
	MiniGPT4-vicuna	MiniGPT4-llama2	2.04	37.24	28.21	48.65	40.48	44.19	0.00	28.98	90.20	71.22
	InstructBLIP	MiniGPT4-llama2	8.16	30.98	56.41	42.42	49.02	45.51	0.00	25.30	90.57	52.57
	BLIP-2	MiniGPT4-llama2	10.20	20.33	53.85	46.73	88.10	61.34	0.00	12.67	96.08	65.72
	BLIVA	MiniGPT4-llama2	6.12	30.72	56.41	42.95	66.67	45.12	0.00	28.76	91.18	53.03
	Qwen2.5-VL	Qwen2-VL	0.00	33.80	20.51	39.30	26.19	37.30	0.00	30.24	12.75	38.74
ImageNet	MiniGPT4-llama2	MiniGPT4-vicuna	2.04	28.61	22.73	32.41	20.37	22.44	10.53	32.28	27.18	32.63
	MiniGPT4-vicuna	MiniGPT4-llama2	10.20	29.32	56.82	42.89	45.45	33.18	17.54	29.92	69.90	51.01
	InstructBLIP	MiniGPT4-llama2	6.12	20.84	47.73	30.76	81.82	36.46	0.00	28.62	94.17	37.56
	BLIP-2	MiniGPT4-llama2	8.16	19.71	56.82	41.76	94.55	50.17	0.00	13.29	93.20	54.17
	BLIVA	MiniGPT4-llama2	4.08	20.27	47.73	28.18	79.21	37.33	5.80	32.88	94.17	36.08
	Qwen2.5-VL	Qwen2-VL	13.95	18.89	36.36	22.56	67.27	29.49	12.20	31.11	29.41	21.87
TextVQA	MiniGPT4-llama2	MiniGPT4-vicuna	7.69	41.43	17.65	45.12	41.67	55.34	9.30	41.53	52.94	51.89
	MiniGPT4-vicuna	MiniGPT4-llama2	14.29	40.73	35.29	48.42	31.25	43.65	4.88	34.28	51.82	51.51
	InstructBLIP	MiniGPT4-vicuna	20.41	39.04	25.49	42.30	77.08	64.74	6.98	37.09	82.35	44.93
	BLIP-2	MiniGPT4-vicuna	12.25	29.50	13.73	26.19	70.83	45.87	4.65	18.09	81.37	54.26
	BLIVA	InstructBLIP	18.18	39.16	23.53	43.92	77.08	47.36	5.13	30.70	88.24	53.51
	Qwen2.5-VL	Qwen2-VL	3.03	42.33	7.84	46.28	6.25	42.25	2.44	37.40	20.59	37.99
Average			8.24	31.15	34.58	39.50	54.57	42.93	4.41	29.01	66.36	47.33

Table 2: Imperceptibility comparison across MSCOCO, ImageNet, and TextVQA. Perturbations are optimized on MiniGPT4-llama2. All metrics are multiplied by 100 for readability. The best results are highlighted in bold font.

Attacks	MSCOCO					ImageNet					TextVQA				
	SSIM↑	MS-SSIM↑	FSIM↑	HaarPSI↑	LPIPS↓	SSIM↑	MS-SSIM↑	FSIM↑	HaarPSI↑	LPIPS↓	SSIM↑	MS-SSIM↑	FSIM↑	HaarPSI↑	LPIPS↓
ARE-W	59.41	81.57	65.88	93.73	70.48	65.51	90.11	67.22	95.31	43.21	66.53	81.28	63.46	94.58	60.59
ARE-B	51.06	69.22	62.31	92.66	85.70	56.66	80.41	59.03	97.47	39.42	61.88	73.17	59.09	98.25	45.44
CI	53.49	73.56	63.75	93.54	78.31	57.69	84.06	62.34	94.26	55.69	57.65	70.49	61.17	94.14	74.32
ATPI	99.38	99.54	99.62	99.65	1.45	97.96	98.44	98.71	98.79	3.66	99.38	99.49	99.57	99.72	1.72
Ours	73.12	93.92	76.88	98.79	18.44	71.24	93.83	72.58	98.85	20.41	71.27	93.51	70.17	98.85	20.50

evaluation, we only allow image perturbations. ATPI directly inserts a malicious textual instruction into the image and optimizes its rendering attributes, such as color, size, and position, to improve readability and attack effectiveness. For ATPI, we use its original prompt, “Please briefly describe the content and text in the image,” because the model otherwise may not be explicitly encouraged to read the injected text. More details on baseline implementation are provided in Appendix D.

5.2 Attack Performance

5.2.1 Overall Attack Performance. We compare CROSSMPI with four representative prompt injection baselines on three datasets and six target LVLms. Table 1 reports the best source model used for perturbation optimization for each target LVLm, together with the ASR (%) and SS (%). Overall, CROSSMPI achieves the best performance across datasets and target models. Averaged over all settings, CROSSMPI obtains an ASR of 66.36%, outperforming the four baselines by 40.91 percentage points on average. Meanwhile, it achieves an average SS of 47.33, exceeding the baseline average by 11.68 points. These results indicate that CROSSMPI not only redirects

LVLms to attacker-chosen tasks more effectively, but also preserves stronger semantic consistency with the desired target responses.

The advantage of CROSSMPI is particularly clear in the detailed results. For example, on MSCOCO, CROSSMPI achieves high ASR across multiple target models, including 90.20% on MiniGPT4-vicuna, 90.57% on InstructBLIP, 96.08% on BLIP-2, and 91.18% on BLIVA, while also maintaining competitive SS scores. In contrast, ARE-W shows limited transferability, with an average ASR of only 8.24%. This is because ARE-W directly optimizes the target output probability on the white-box source model, the resulting perturbations tend to overfit source-model-specific generation behaviors and transfer poorly to other LVLms. The CLIP-based baselines, ARE-B and CI, achieve higher ASR than ARE-W, with average ASRs of 34.58% and 54.57%, respectively. This indicates that optimizing image embeddings toward target semantics can partially change the model’s visual understanding and induce target-related outputs. However, since the benign text prompt is still interpreted as the original task, these methods are less effective at changing the model’s task-level behavior. Their strong reliance on target-semantic image alignment also introduces more visible perturbations, resulting in weaker imperceptibility, as discussed in Section 5.2.2. ATPI performs

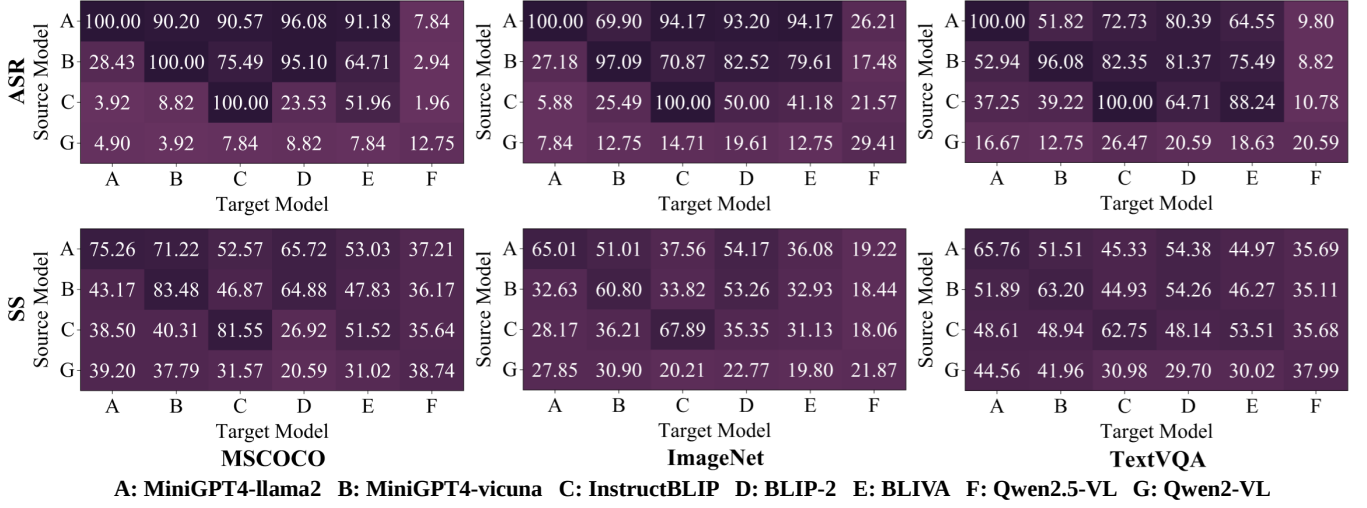


Figure 4: Heatmap of CrossMPI across LVLMS. Perturbations are optimized on MiniGPT4-llama2, MiniGPT4-vicuna, InstructBLIP, and Qwen2-VL, and then evaluated on target models.

the weakest overall, achieving only 4.41% average ASR. Although it embeds malicious textual instructions into the image and uses prompts that encourage the model to read image text, this strategy does not reliably trigger the desired cross-modal instruction behavior. It also makes the injected instruction directly visible to users, which further limits its practicality. In contrast, CrossMPI attacks the internal multimodal reasoning process more directly. By optimizing fusion-critical hidden states and allocating perturbation budgets to semantic-critical image regions, CrossMPI better aligns the perturbation with the LVLMS’s internal task interpretation. This explains its stronger attack effectiveness, better transferability, and improved semantic alignment across diverse models and datasets.

5.2.2 Imperceptibility. Imperceptibility is a key requirement for practical prompt injection attacks, since an attacked image is more likely to be used by a victim when it remains visually close to the original image. Therefore, beyond attack effectiveness, we evaluate whether different methods can preserve the visual fidelity of the input image after applying adversarial perturbation. Figure 9 shows qualitative examples of different attacks. We further quantify imperceptibility using five widely used image-similarity metrics: SSIM [54], MS-SSIM [55], FSIM [69], HaarPSI [39], and LPIPS [70]. SSIM and MS-SSIM evaluate structural consistency, FSIM captures feature-level similarity, HaarPSI measures perceptual similarity based on Haar wavelet responses, and LPIPS estimates perceptual distance in a deep feature space. Higher SSIM, MS-SSIM, FSIM, and HaarPSI indicate better visual preservation, while lower LPIPS indicates smaller perceptual distortion.

As shown in Table 2, ATPI achieves the best scores on most metrics because it adds visible malicious text rather than dense image perturbations. This means that it directly exposes the malicious instruction to the user, which weakens its practicality. Among the remaining noise-optimization attacks, CrossMPI achieves the best imperceptibility. For example, on MSCOCO, CrossMPI obtains an LPIPS score of only 18.44, whereas ARE-W, ARE-B, and CI obtain

70.48, 85.70, and 78.31, respectively. The visual examples in Figure 9 are consistent with these results: the images generated by CrossMPI remain visually close to the originals, while ARE-B and CI often produce noticeable appearance changes. This is because their objectives push image embeddings toward target semantics, causing the attacked images to visually drift toward the target concept. This effect is especially evident for CI, where the optimized images exhibit visible target-related patterns, such as sheep-like patterns on MSCOCO, cat-like patterns on ImageNet, and phone-like patterns on TextVQA. In contrast, CrossMPI allocates larger perturbation budgets to semantic-critical regions and optimizes fusion-critical hidden states, avoiding direct modification of the image semantics. In addition, we use $\mathcal{L}_{freq}$ to suppress high-frequency components in the perturbation, which further improves visual imperceptibility.

5.2.3 Cross-Source Validation. We further evaluate CrossMPI under different source model settings to examine its transferability. Specifically, we use MiniGPT4-llama2, MiniGPT4-vicuna, InstructBLIP, and Qwen2-VL as source models to optimize image perturbations, and then evaluate the generated attacked images on six target LVLMS. This setting includes both the white-box case, where the source and target models are the same, and the transfer case, where the target model is different from the source model. Figure 4 shows the results of CrossMPI across different source-target pairs, and Table 8 provides the detailed comparison with baseline attacks.

The results show that CrossMPI can achieve 99.24% white-box attack success rates and can also transfer successfully to a broad range of target models. For example, on MSCOCO, the attacked images optimized on MiniGPT4-llama2 can reach 90.20%, 90.57%, 96.08%, and 91.18% ASR on MiniGPT4-vicuna, InstructBLIP, BLIP-2, and BLIVA, respectively. These attacks also maintain strong semantic alignment with the target responses, with SS scores of 71.22%, 52.57%, 65.72%, and 53.03%. The results also demonstrate that transferability is stronger between models with more similar architectures. For instance, on TextVQA, attacked images optimized

on MiniGPT4-llama2, MiniGPT4-vicuna, and InstructBLIP achieve only 9.80%, 8.82%, and 10.78% ASR on Qwen2.5-VL, respectively. In contrast, images optimized on Qwen2-VL achieve 20.59% ASR on Qwen2.5-VL. This is expected as models with similar architectures tend to share more comparable model hidden state spaces and fusion mechanisms, making perturbations optimized on one model more likely to change the internal task interpretation of another. In other words, when attackers can access a source model with the same architecture type as the target model, CrossMPI can achieve prompt injection attacks with high success rates.

5.3 Ablation Study

5.3.1 Layer Selection for Optimization. To examine which model layers provide the most effective optimization target, we conduct an ablation study on layer selection. We use MiniGPT4-llama2 as the source model and do not apply the perturbation budget mask. We define three layer groups: early layers, middle layers, and final layers. Based on the fusion-layer analysis in Figure 6, we use the selected fusion-critical layers as the middle-layer group. Specifically, the three groups are early layers {1, 2, 3}, fusion-critical middle layers {12, 13, 14}, and final layers {30, 31, 32}. We then evaluate six optimization settings, including optimizing each group alone and optimizing different combinations of two groups.

The results show that attacks optimized on the middle fusion-critical layers achieve the best performance. Compared with early or final layers, optimizing these layers yields the most consistent attack performance across different LVLs. For example, fusion-layer optimization achieves an ASR of 90.16% on MiniGPT4-vicuna, compared with 89.20% for early-layer optimization and 87.25% for final-layer optimization. This indicates that perturbations become less effective when applied before or after the main multimodal fusion stage of LVLs. This finding is consistent with the functional differences among LVL layers. Early layers mainly process modality-specific visual signals, so perturbations at this stage may not directly affect the model’s task understanding. Final layers operate after the model has already formed a prompt-conditioned representation, making them more tied to model-specific response generation. In contrast, fusion-critical layers lie at the point where the image and the text prompt are integrated into a task representation. Perturbing these layers can therefore more directly affect the model’s task interpretation, leading to stronger and more stable attack performance.

5.3.2 Effect of the Perturbation Budget Mask. To examine the role of perturbation budget allocation, we conduct an ablation study with different masking strategies. We use MiniGPT4-llama2 as the source model and optimize the same fusion-critical layers in all settings. We compare three budget allocation strategies: using no mask, applying the saliency-only mask adopted in SGMA [71], and using our distance-decremental budget mask with different coefficients. This comparison allows us to evaluate whether focusing perturbations only on highly salient regions is sufficient, or whether a spatially smoother budget allocation leads to more stable transfer across target models.

Table 4 reports the results for $\lambda \in \{0.1, 0.2, 0.3, 0.4, 0.5\}$, and the results for the remaining coefficients are provided in Table 7.

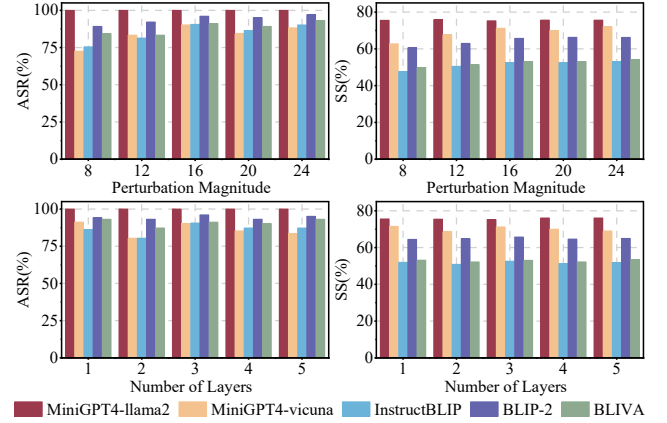


Figure 5: Ablation of perturbation budget and optimized layer count on MSCOCO. The first row varies $\epsilon \in \{8, 12, 16, 20, 24\}/255$, and the second row varies the number of optimized fusion-critical layers from one to five, using MiniGPT4-llama2 as the source model.

Overall, $\lambda = 0.3$ gives the best trade-off between concentrating perturbations on important regions and preserving sufficient spatial coverage. It achieves the highest ASR on MiniGPT4-vicuna, InstructBLIP, and BLIVA, reaching 90.20%, 90.57%, and 91.18%, respectively. As shown in the mask visualizations in Figure 10, increasing λ leads to a more pronounced step-wise decrease in the perturbation budget. The trend across different coefficients shows that perturbation allocation should be neither too uniform nor too concentrated. When λ is too small, the mask is close to uniform allocation, so the optimization does not sufficiently emphasize semantically important regions. When λ is too large, most of the perturbation budget is restricted to a small set of pixels, which weakens the attack’s ability to jointly affect local visual evidence and broader contextual information. This confirms that a moderate distance-decremental mask is more effective for cross-model transfer. Based on these results, we set $\lambda = 0.3$ in the final configuration.

5.3.3 Effect of the Perturbation Budget. The perturbation budget directly controls the maximum visual modification allowed for the attack: a larger budget expands the feasible optimization region, but can also reduce visual stealthiness. We evaluate five perturbation budgets, $\epsilon \in \{8, 12, 16, 20, 24\}/255$, on MSCOCO using MiniGPT4-llama2 as the source model, while keeping all other optimization settings unchanged. As shown in the first row of Figure 5, CrossMPI remains effective across all perturbation budgets. Even with the smallest budget, $\epsilon = 8/255$, the attack still achieves over 70% ASR on all target models, showing that CrossMPI can succeed under a strict perturbation constraint. As ϵ increases, the ASR generally improves across models. This trend is expected because a larger budget allows the optimization to more flexibly modify the fusion-layer representations toward the attacker-specified task. These results show that CrossMPI does not rely on excessively large perturbations, while a larger budget can further improve attack success by providing more room for controlled semantic manipulation.

Table 3: Ablation of layer-selection strategies for optimization. ASR (%) and SS (%) are reported across target LVLs for different layer groups, with perturbations optimized on MiniGPT4-llama2. The best results are highlighted in bold font.

Layer Group	MiniGPT4-llama2		MiniGPT4-vicuna		InstructBLIP		BLIP-2		BLIVA	
	ASR	SS	ASR	SS	ASR	SS	ASR	SS	ASR	SS
early	100	76.01	89.20	72.21	88.20	52.77	95.56	67.46	90.36	52.45
fusion	100	75.17	90.16	72.63	89.18	52.87	96.08	65.33	90.12	53.85
final	100	77.58	87.25	72.89	88.65	52.86	96.78	66.34	90.19	54.56
early+fusion	100	76.07	88.24	72.31	85.29	51.99	95.54	67.51	91.18	53.68
early+final	100	75.57	89.18	72.88	88.33	53.66	97.14	67.71	92.10	54.55
fusion+final	100	76.23	88.24	72.58	87.25	53.05	96.36	67.47	92.10	54.35

Table 4: Ablation of different perturbation budget masks for optimization. ASR (%) and SS (%) are reported across target LVLs for no mask, saliency-only mask, and CrossMPI masks with $\lambda \leq 0.5$, using perturbations optimized on MiniGPT4-llama2. The best results are highlighted in bold font.

Budget Mask	MiniGPT4-llama2		MiniGPT4-vicuna		InstructBLIP		BLIP-2		BLIVA	
	ASR	SS	ASR	SS	ASR	SS	ASR	SS	ASR	SS
no mask	100	75.17	90.16	72.63	89.18	52.87	96.08	65.33	90.12	53.85
saliency-only	100	75.49	83.33	69.90	85.29	51.36	94.12	64.89	88.24	52.65
$\lambda=0.1$	100	75.42	86.27	69.63	88.57	51.86	93.14	65.12	87.25	51.63
$\lambda=0.2$	100	75.21	88.24	71.28	87.62	51.76	94.12	65.17	87.25	52.00
$\lambda=0.3$	100	75.26	90.20	71.22	90.57	52.57	96.08	65.72	91.18	53.03
$\lambda=0.4$	100	75.30	83.33	70.51	87.25	51.38	93.14	64.68	89.22	52.58
$\lambda=0.5$	100	75.53	86.27	71.85	82.35	51.29	94.12	64.98	89.22	52.77

5.3.4 Effect of the Number of Selected Layers. The number of selected layers controls how much of the multimodal fusion process is directly constrained during optimization. Optimizing too few layers may provide insufficient coverage of the fusion stage, while optimizing too many layers may introduce less relevant or more model-specific representations. We therefore evaluate the effect of the number of optimized fusion-critical layers. Specifically, we vary the number of optimized layers from 1 to 5, using MiniGPT4-llama2 as the source model on MSCOCO. As shown in the second row of Figure 5, CrossMPI maintains strong attack performance across different layer counts, showing that it is not sensitive to the exact number of selected layers within the fusion-critical region. Among all settings, optimizing three layers achieves the best overall performance. This suggests that three fusion-critical layers provide enough coverage of task-semantic fusion while keeping the optimization target focused. Increasing the number of optimized layers to four or five does not further improve performance, likely because the additional layers are less directly related to multimodal fusion and may introduce model-specific optimization noise.

5.4 Defenses

To defend against our attack, we suggest adopting defense methods that either transform the input image before inference or strengthen the inference procedure itself. In our experiments, we evaluate five representative defenses from these two categories. The first category is *input-transformation defenses*, which preprocess the image to disrupt adversarial perturbations. It includes Randomization [8, 60], which applies random resizing and padding; RandomRotation [20, 25], which introduces small random geometric transformations; and JPEG Compression [10], which suppresses

high-frequency perturbation artifacts through image recompression. The second category is *inference-safeguarding defenses*, which modify the prediction process rather than the input alone. It includes SmoothVLM [46], which aggregates predictions over randomly masked image variants, and DPS [74], which uses partial image observations and a correction prompt to guide the final response. Detailed instantiations are provided in Appendix D.

Table 5 evaluates the robustness of CrossMPI under different defenses, where perturbations are optimized on MiniGPT4-llama2. Additional results are provided in Appendix E.3. Overall, these defenses reduce the attack success rate to varying degrees, but none of them completely eliminates the attack. Among the evaluated defenses, SmoothVLM is the most effective in most settings. It reduces the ASR to below 5% on MSCOCO and ImageNet, and keeps the ASR below 10% on TextVQA across all target models. JPEG Compression also consistently weakens the attack, indicating that part of the optimized perturbation is sensitive to frequency-domain degradation. In comparison, Randomization and RandomRotation provide only limited protection, suggesting that the perturbations can still survive mild spatial transformations. DPS shows less stable defense performance, which is less effective on the source model and in several transfer settings. For example, under DPS, CrossMPI still achieves 52.94% ASR on MSCOCO and 71.84% ASR on ImageNet. It also reaches 65.05% ASR on BLIVA for ImageNet. These results suggest that existing defenses can mitigate cross-modal prompt injection attacks, but stronger defense mechanisms are still needed.

Table 5: Defense evaluation for perturbations optimized on MiniGPT4-llama2. ASR (%) and SS (%) are reported across datasets, target LVLMs, and defense strategies.

Datasets	Defenses	MiniGPT4-llama2		MiniGPT4-vicuna		InstructBLIP		BLIP-2		BLIVA	
		ASR	SS	ASR	SS	ASR	SS	ASR	SS	ASR	SS
MSCOCO	Randomization	13.73	41.83	4.90	38.52	17.65	33.47	24.51	45.71	21.57	34.53
	RandomRotation	22.55	44.80	9.80	39.57	16.67	34.38	18.63	43.68	19.61	34.23
	JPEG Compression	12.75	41.20	5.88	37.60	8.82	32.05	8.82	39.71	10.78	31.58
	SmoothVLM	0.00	37.43	0.00	37.03	0.98	30.43	0.98	37.60	0.98	30.02
	DPS	52.94	55.14	32.35	48.50	14.71	32.69	0.00	34.42	47.06	39.73
ImageNet	Randomization	31.07	38.25	16.50	31.89	33.98	23.80	38.83	37.26	28.16	23.05
	RandomRotation	33.01	40.63	22.33	32.87	34.95	24.00	29.13	35.39	27.18	23.64
	JPEG Compression	17.48	33.96	17.48	31.92	19.42	22.29	19.42	32.28	16.50	21.24
	SmoothVLM	0.97	26.44	0.97	28.46	3.88	20.07	0.97	27.84	1.94	19.44
	DPS	71.84	50.67	30.10	35.85	21.36	24.68	6.80	36.60	65.05	33.18
TextVQA	Randomization	32.73	47.44	19.61	42.86	35.29	33.19	12.75	30.53	29.41	31.40
	RandomRotation	30.91	47.71	17.65	42.81	32.35	32.12	14.71	30.87	26.47	32.38
	JPEG Compression	20.59	43.56	15.53	43.88	24.51	30.92	6.86	28.12	21.57	29.66
	SmoothVLM	5.45	41.18	5.83	41.45	8.82	28.90	1.96	28.31	9.80	30.69
	DPS	55.45	50.62	14.71	44.15	24.51	34.84	0.00	30.32	31.37	36.68

6 Related Work

6.1 Prompt Injection Attacks

Prompt injection attacks manipulate LLM behavior by injecting malicious instructions into the model input, causing the model to deviate from the intended task and follow attacker-chosen objectives. Early studies mainly focus on manually designed prompt injection strategies. These works show that simple injected instruction sequences can already be effective, especially when they are concatenated with benign data or separated using special characters such as newlines (“\n”) and tabs (“\t”) [11, 56]. Another common strategy is to use explicit task-overriding instructions, such as “Ignore my previous instructions ...”, to make the model discard the original task context and execute the injected task instead [3, 11, 33, 56]. Some attacks further inject fake responses to the original task, misleading the model into treating the benign task as completed and continuing with the malicious instruction [57]. Based on these attack patterns, Liu *et al.* [23] formalize prompt injection attacks and combine multiple manual strategies to improve attack effectiveness.

Recent studies also investigate automated attacks under white-box access. Greedy Coordinate Gradient (GCG) [76] uses gradient information to iteratively search for adversarial input tokens that increase the likelihood of a target model response. Liu *et al.* [22] apply GCG to prompt injection by optimizing injected instruction strings according to different model response behaviors. Shi *et al.* [43] study prompt injection in the LLM-as-a-Judge setting, where the model selects the best answer from multiple candidates. In this scenario, attackers embed optimized injection strings into a controlled candidate response, causing the judge model to favor the attacker-controlled answer regardless of its actual quality. These works demonstrate that prompt injection can be generated both manually and automatically, posing a practical threat to instruction-following LLM systems.

6.2 Attacks on LVLMs

With the rapid development of LVLMs, their security risks have attracted increasing attention. Existing studies have explored several types of attacks against LVLMs, including prompt injection, adversarial, jailbreak, membership inference, and backdoor attacks.

For prompt injection attacks, Wu *et al.* [58] craft image-based injections by using multiple CLIP models [36] as surrogate models, encouraging the attacked image to align with the target response. Li *et al.* [16] embed malicious instructions directly into images and adjust visual factors such as position, size, and color so that LVLMs can recognize the injected text during inference. Wang *et al.* [51] optimize the visual embeddings toward target-image semantics while also optimizing deceptive text prompts with a source model.

Beyond prompt injection, many works study adversarial attacks that aim to induce specific LVLm responses. Wang *et al.* [50] optimize modality-consistency features with attention-guided perturbations and use orthogonality constraints to improve cross-model transferability. Cai *et al.* [4] formulate the attack from an information-theoretic perspective by maximizing the mutual information between the perturbation and the target response while reducing the influence of the original image content. Li *et al.* [18] propose MABA, an untargeted black-box attack that disrupts both visual encoding and modality alignment in LVLMs by suppressing discriminative visual features and using a mutual-information-aware projector to simulate the cross-modal adapter.

Other studies investigate broader security threats to LVLMs. Jailbreak attacks [42, 66] craft harmful multimodal inputs to bypass safety mechanisms and induce unsafe responses. Qi *et al.* [35] optimize adversarial images in a white-box setting to increase the likelihood of malicious outputs, while Yang *et al.* [64] decompose harmful queries into sub-questions and construct contrastive sub-images to distract model attention from safety constraints. For membership inference, Li *et al.* [17] propose a cross-modal inference pipeline and use a MaxRényi-K% metric based on output logits to identify whether an image or text sample appears in the training set. Hu *et al.* [13] exploit LVLMs’ sensitivity to temperature

parameter changes to distinguish members from non-members. For backdoor attacks, Lyu *et al.* [27] inject trigger patterns using out-of-distribution data and combine knowledge distillation with semantic consistency constraints, while Liu *et al.* [24] design structured low-frequency triggers during the self-supervised pretraining stage of LVLMs to make backdoor patterns better aligned with natural image features.

7 Conclusion & Future Work

In this paper, we propose CROSSMPI, a novel cross-modal prompt injection attack against LVLMs. CROSSMPI uses image-only perturbations to change the model's interpretation of both the visual input and the textual prompt, causing the model to execute an attacker-chosen task while the text prompt remains unchanged. We explore a new direction for prompt injection optimization by optimizing in the model hidden state space instead of the embedding space, where visual and textual information are integrated. To reduce the large optimization space, we introduce two strategies: fusion-critical layer selection and distance-decremental perturbation budget assignment. The first strategy reduces the model parameter space by restricting optimization to a small set of layers most relevant to multimodal fusion. We further find that the most effective layers are located in the middle of LVLMs, especially the fusion-critical layers, which differs from the common practice of optimizing final layers in adversarial perturbation methods. The second strategy reduces the image search space by concentrating the perturbation budget around semantic-critical regions. Experiments across multiple datasets and LVLMs demonstrate the effectiveness, transferability, and imperceptibility of CROSSMPI. A current limitation is that the attack still depends on specific text prompts. In future work, our objective is to explore prompt-agnostic image-only attacks and develop defenses against such prompt injection.

References

- [1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altmenschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774* (2023).
- [2] Jean-Baptiste Alayrac, Jeff Donahue, Pauline Luc, Antoine Miech, Iain Barr, Yana Hasson, Karel Lenc, Arthur Mensch, Katherine Millican, Malcolm Reynolds, et al. 2022. Flamingo: a visual language model for few-shot learning. *Advances in neural information processing systems* 35 (2022), 23716–23736.
- [3] Hezekiah J Branch, Jonathan Rodriguez Cefalu, Jeremy McHugh, Leyla Hujer, Aditya Bahl, Daniel del Castillo Iglesias, Ron Heichman, and Ramesh Darwishi. 2022. Evaluating the susceptibility of pre-trained language models via handcrafted adversarial examples. *arXiv preprint arXiv:2209.02128* (2022).
- [4] Xiaowen Cai, Daizong Liu, Xiaoye Qu, Xiang Fang, Jianfeng Dong, Keke Tang, Pan Zhou, Lichao Sun, and Wei Hu. 2025. Towards Building Model/Prompt-Transferable Attackers against Large Vision-Language Models. In *The Thirtieth Annual Conference on Neural Information Processing Systems*.
- [5] Lin Chen, Jinsong Li, Xiaoyi Dong, Pan Zhang, Conghui He, Jiaqi Wang, Feng Zhao, and Dahua Lin. 2024. Sharegpt4v: Improving large multi-modal models with better captions. In *European Conference on Computer Vision*. Springer, 370–387.
- [6] Xiangxiang Chen, Peixin Zhang, Jun Sun, Wenhui Wang, and Jingyi Wang. 2025. Rounding-Guided Backdoor Injection in Deep Learning Model Quantization. *arXiv preprint arXiv:2510.09647* (2025).
- [7] Wenliang Dai, Junnan Li, Dongxu Li, Anthony Tiong, Junqi Zhao, Weisheng Wang, Boyang Li, Pascale N Fung, and Steven Hoi. 2023. Instructblip: Towards general-purpose vision-language models with instruction tuning. *Advances in neural information processing systems* 36 (2023), 49250–49267.
- [8] Iuri Frosio and Jan Kautz. 2023. The best defense is a good offense: Adversarial augmentation against adversarial attacks. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 4067–4076.
- [9] Yash Goyal, Tejas Khot, Douglas Summers-Stay, Dhruv Batra, and Devi Parikh. 2017. Making the v in vqa matter: Elevating the role of image understanding in visual question answering. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 6904–6913.
- [10] Chuan Guo, Mayank Rana, Moustapha Cisse, and Laurens van der Maaten. 2018. Countering Adversarial Images using Input Transformations. In *International Conference on Learning Representations*.
- [11] Rich Harang. 2023. Securing LLM Systems Against Prompt Injection. <https://developer.nvidia.com/blog/securing-llm-systems-against-prompt-injection>
- [12] Wenbo Hu, Yifan Xu, Yi Li, Weiyue Li, Zeyuan Chen, and Zhuowen Tu. 2024. Bliva: A simple multimodal llm for better handling of text-rich visual questions. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38. 2256–2264.
- [13] Yuke Hu, Zheng Li, Zhihao Liu, Yang Zhang, Zhan Qin, Kui Ren, and Chun Chen. 2025. Membership inference attacks against vision-language models. *arXiv preprint arXiv:2501.18624* (2025).
- [14] Andrey Labunets, Nishit V Pandya, Ashish Hooda, Xiaohan Fu, and Earlene Fernandes. 2025. Fun-tuning: Characterizing the vulnerability of proprietary llms to optimization-based prompt injection attacks via the fine-tuning interface. In *2025 IEEE Symposium on Security and Privacy (SP)*. IEEE, 411–429.
- [15] Junnan Li, Dongxu Li, Silvio Savarese, and Steven Hoi. 2023. Blip-2: Bootstrapping language-image pre-training with frozen image encoders and large language models. In *International conference on machine learning*. PMLR, 19730–19742.
- [16] Yanjie Li, Yiming Cao, Dong Wang, and Bin Xiao. 2025. AgentTypo: Adaptive Typographic Prompt Injection Attacks against Black-box Multimodal Agents. *arXiv preprint arXiv:2510.04257* (2025).
- [17] Zhan Li, Yongtao Wu, Yihang Chen, Francesco Tonin, Elias Abad Rocamora, and Volkan Cevher. 2024. Membership inference attacks against large vision-language models. *Advances in Neural Information Processing Systems* 37 (2024), 98645–98674.
- [18] Zhichao Li, Hongshan Yang, Zhibo Wang, Huiyu Xu, Junhong Lai, Yaopeng Wang, Kui Ren, and Chun Chen. [n. d.]. On Evaluating the Robustness of Large Vision-Language Models via Untargeted Modality Alignment Breaking Adversarial Attack. ([n. d.]).
- [19] Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C Lawrence Zitnick. 2014. Microsoft coco: Common objects in context. In *European conference on computer vision*. Springer, 740–755.
- [20] Daizong Liu, Mingyu Yang, Xiaoye Qu, Pan Zhou, Xiang Fang, Keke Tang, Yao Wan, and Lichao Sun. 2024. Pandora's box: Towards building universal attackers against real-world large vision-language models. *Advances in Neural Information Processing Systems* 37 (2024), 52127–52158.
- [21] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. 2023. Visual instruction tuning. *Advances in neural information processing systems* 36 (2023), 34892–34916.
- [22] Xiaogeng Liu, Zhiyuan Yu, Yizhe Zhang, Ning Zhang, and Chaowei Xiao. 2024. Automatic and universal prompt injection attacks against large language models. *arXiv preprint arXiv:2403.04957* (2024).
- [23] Yupei Liu, Yuqi Jia, Rumpeng Geng, Jinyuan Jia, and Neil Zhenqiang Gong. 2024. Formalizing and benchmarking prompt injection attacks and defenses. In *33rd USENIX Security Symposium (USENIX Security 24)*. 1831–1847.
- [24] Zhaoyi Liu and Huan Zhang. 2025. Stealthy Backdoor Attack in Self-Supervised Learning Vision Encoders for Large Vision Language Models. In *Proceedings of the Computer Vision and Pattern Recognition Conference*. 25060–25070.
- [25] Haochen Luo, Jindong Gu, Fengyuan Liu, and Philip Torr. 2024. An image is worth 1000 lies: Adversarial transferability across prompts on vision-language models. *arXiv preprint arXiv:2403.09766* (2024).
- [26] Peizhuo Lv, Chang Yue, Ruigang Liang, Yunfei Yang, Shengzhi Zhang, Hualong Ma, and Kai Chen. 2023. A data-free backdoor injection approach in neural networks. In *32nd USENIX Security Symposium (USENIX Security 23)*. 2671–2688.
- [27] Weimin Lyu, Jiachen Yao, Saumya Gupta, Lu Pang, Tao Sun, Lingjie Yi, Lijie Hu, Haibin Ling, and Chao Chen. [n. d.]. Backdooring Vision-Language Models with Out-Of-Distribution Data. In *The Thirteenth International Conference on Learning Representations*.
- [28] Clement Neo, Luke Ong, Philip Torr, Mor Geva, David Krueger, and Fazl Barez. 2025. Towards Interpreting Visual Information Processing in Vision-Language Models. In *The Thirteenth International Conference on Learning Representations*.
- [29] Alexander Quinn Nichol, Prafulla Dhariwal, Aditya Ramesh, Pranav Shyam, Pamela Mishkin, Bob McGrew, Ilya Sutskever, and Mark Chen. 2022. GLIDE: Towards Photorealistic Image Generation and Editing with Text-Guided Diffusion Models. In *International Conference on Machine Learning*. PMLR, 16784–16804.
- [30] Nostalgebraist. 31st Aug 2020. Interpreting GPT: The logit lens. <https://www.alignmentforum.org/posts/AcKRB8wDpdaN6v6ru/interpreting-gpt-the-logit-lens>
- [31] OWASP. 2023. OWASP Top 10 for Large Language Model Applications. https://owasp.org/www-project-top-10-for-large-language-model-applications/assets/PDF/OWASP-Top-10-for-LLMs-2023-v1_1.pdf
- [32] Dario Pasquini, Martin Strohmeier, and Carmela Troncoso. 2024. Neural exec: Learning (and learning from) execution triggers for prompt injection attacks. In *Proceedings of the 2024 Workshop on Artificial Intelligence and Security*. 89–100.

- [33] Fábio Perez and Ian Ribeiro. 2022. Ignore previous prompt: Attack techniques for language models. *arXiv preprint arXiv:2211.09527* (2022).
- [34] Julien Piet, Maha Alrashed, Chawin Sitawarin, Sizhe Chen, Zeming Wei, Elizabeth Sun, Basel Alomair, and David Wagner. 2024. Jatmo: Prompt injection defense by task-specific finetuning. In *European Symposium on Research in Computer Security*. Springer, 105–124.
- [35] Xiangyu Qi, Kaixuan Huang, Ashwinee Panda, Peter Henderson, Mengdi Wang, and Prateek Mittal. 2024. Visual adversarial examples jailbreak aligned large language models. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 38. 21527–21536.
- [36] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. 2021. Learning transferable visual models from natural language supervision. In *International conference on machine learning*. PMLR, 8748–8763.
- [37] Aditya Ramesh, Prafulla Dhariwal, Alex Nichol, Casey Chu, and Mark Chen. 2022. Hierarchical text-diffusion image generation with clip latents. *arXiv preprint arXiv:2204.06125* 1, 2 (2022), 3.
- [38] Nils Reimers and Iryna Gurevych. 2019. Sentence-bert: Sentence embeddings using siamese bert-networks. In *Proceedings of the 2019 conference on empirical methods in natural language processing and the 9th international joint conference on natural language processing (EMNLP-IJCNLP)*. 3982–3992.
- [39] Rafael Reisenhofer, Sebastian Bosse, Gitta Kutyniok, and Thomas Wiegand. 2018. A Haar wavelet-based perceptual similarity index for image quality assessment. *Signal Processing: Image Communication* 61 (2018), 33–43.
- [40] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. 2022. High-resolution image synthesis with latent diffusion models. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 10684–10695.
- [41] Olga Russakovsky, Jia Deng, Hao Su, Jonathan Krause, Sanjeev Satheesh, Sean Ma, Zhiheng Huang, Andrej Karpathy, Aditya Khosla, Michael Bernstein, et al. 2015. Imagenet large scale visual recognition challenge. *International journal of computer vision* 115, 3 (2015), 211–252.
- [42] Xinyue Shen, Zeyuan Chen, Michael Backes, Yun Shen, and Yang Zhang. 2024. "do anything now": Characterizing and evaluating in-the-wild jailbreak prompts on large language models. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*. 1671–1685.
- [43] Jiawen Shi, Zenghui Yuan, Yinyao Liu, Yue Huang, Pan Zhou, Lichao Sun, and Neil Zhenqiang Gong. 2024. Optimization-based prompt injection attack to llm-as-a-judge. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*. 660–674.
- [44] Amanpreet Singh, Vivek Natarajan, Meet Shah, Yu Jiang, Xinlei Chen, Dhruv Batra, Devi Parikh, and Marcus Rohrbach. 2019. Towards vqa models that can read. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 8317–8326.
- [45] Dawn Song, Kevin Eykholt, Ivan Evtimov, Earlene Fernandes, Bo Li, Amir Rahmati, Florian Tramer, Atul Prakash, and Tadayoshi Kohno. 2018. Physical adversarial examples for object detectors. In *12th USENIX workshop on offensive technologies (WOOT 18)*.
- [46] Jiachen Sun, Changsheng Wang, Jiong Xiao Wang, Yiwei Zhang, and Chaowei Xiao. 2024. Safeguarding vision-language models against patched visual prompt injectors. *arXiv preprint arXiv:2405.10529* (2024).
- [47] Qwen Team. 2025. Qwen2.5-VL. <https://qwenlm.github.io/blog/qwen2.5-vl/>
- [48] Sam Toyer, Olivia Watkins, Ethan Adrian Mendes, Justin Sveglato, Luke Bailey, Tiffany Wang, Isaac Ong, Karim Elmaaroufi, Pieter Abbeel, Trevor Darrell, et al. [n.d.]. Tensor Trust: Interpretable Prompt Injection Attacks from an Online Game. In *The Twelfth International Conference on Learning Representations*.
- [49] Maria Tsimpoukelli, Jacob L Menick, Serkan Cabi, SM Eslami, Oriol Vinyals, and Felix Hill. 2021. Multimodal few-shot learning with frozen language models. *Advances in Neural Information Processing Systems* 34 (2021), 200–212.
- [50] Haodi Wang, Kai Dong, Zhilei Zhu, Haotong Qin, Aishan Liu, Xiaolin Fang, Jiakai Wang, and Xianglong Liu. 2024. Transferable multimodal attack on vision-language pre-training models. In *2024 IEEE Symposium on Security and Privacy (SP)*. IEEE, 1722–1740.
- [51] Le Wang, Zonghao Ying, Tianyuan Zhang, Siyuan Liang, Shengshan Hu, Mingchuan Zhang, Aishan Liu, and Xianglong Liu. 2025. Manipulating multimodal agents via cross-modal prompt injection. In *Proceedings of the 33rd ACM International Conference on Multimedia*. 10955–10964.
- [52] Peng Wang, Shuai Bai, Sinan Tan, Shijie Wang, Zhihao Fan, Jinze Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Yang Fan, Kai Dang, Mengfei Du, Xuancheng Ren, Rui Men, Dayiheng Liu, Chang Zhou, Jingren Zhou, and Junyang Lin. 2024. Qwen2-VL: Enhancing Vision-Language Model's Perception of the World at Any Resolution. *arXiv preprint arXiv:2409.12191* (2024).
- [53] Weihang Wang, Qingsong Lv, Wenmeng Yu, Wenyi Hong, Ji Qi, Yan Wang, Junhui Ji, Zhuoyi Yang, Lei Zhao, Song XiXuan, et al. 2024. Cogvlm: Visual expert for pretrained language models. *Advances in Neural Information Processing Systems* 37 (2024), 121475–121499.
- [54] Zhou Wang, Alan C Bovik, Hamid R Sheikh, and Eero P Simoncelli. 2004. Image quality assessment: from error visibility to structural similarity. *IEEE transactions on image processing* 13, 4 (2004), 600–612.
- [55] Zhibo Wang, Hengchang Guo, Zhifei Zhang, Wenxin Liu, Zhan Qin, and Kui Ren. 2021. Feature importance-aware transferable adversarial attacks. In *Proceedings of the IEEE/CVF international conference on computer vision*. 7639–7648.
- [56] Simon Willison. 2022. Prompt injection attacks against GPT-3. <https://simonwillison.net/2022/Sep/12/prompt-injection/>
- [57] Simon Willison. 2023. Delimiters won't save you from prompt injection. <https://simonwillison.net/2023/May/11/delimiters-wont-save-you/>
- [58] Chen Henry Wu, Rishi Rajesh Shah, Jing Yu Koh, Russ Salakhutdinov, Daniel Fried, and Aditi Raghunathan. [n.d.]. Dissecting Adversarial Robustness of Multimodal LM Agents. In *The Thirteenth International Conference on Learning Representations*.
- [59] Shengqiong Wu, Hao Fei, Leigang Qu, Wei Ji, and Tat-Seng Chua. 2024. Next-gpt: Any-to-any multimodal llm. In *Forty-first International Conference on Machine Learning*.
- [60] Cihang Xie, Jianyu Wang, Zhishuai Zhang, Zhou Ren, and Alan Yuille. 2018. Mitigating Adversarial Effects Through Randomization. In *International Conference on Learning Representations*.
- [61] Chejian Xu, Mintong Kang, Jiawei Zhang, Zeyi Liao, Lingbo Mo, Mengqi Yuan, Huan Sun, and Bo Li. 2024. Advweb: Controllable black-box attacks on vlm-powered web agents. (2024).
- [62] Hai Yan, Haijian Ma, Xiaowen Cai, Daizong Liu, Zenghui Yuan, Xiaoye Qu, Jianfeng Dong, Runwei Guan, Xiang Fang, Hongyang He, et al. 2025. Fit the Distribution: Cross-Image/Prompt Adversarial Attacks on Multimodal Large Language Models. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*.
- [63] Dingchen Yang, Bowen Cao, Anran Zhang, Weibo Gu, Winston Hu, and Guang Chen. 2025. Beyond intermediate states: Explaining visual redundancy through language. *arXiv preprint arXiv:2503.20540* (2025).
- [64] Zuopeng Yang, Juluan Fan, Anli Yan, Erdun Gao, Xin Lin, Tao Li, Kanghua Mo, and Changyu Dong. 2025. Distraction is all you need for multimodal large language model jailbreaking. In *Proceedings of the Computer Vision and Pattern Recognition Conference*. 9467–9476.
- [65] Zhuoran Yu and Yong Jae Lee. 2025. How Multimodal LLMs Solve Image Tasks: A Lens on Visual Grounding, Task Reasoning, and Answer Decoding. In *Second Conference on Language Modeling*.
- [66] Zhiyuan Yu, Xiaogeng Liu, Shunning Liang, Zach Cameron, Chaowei Xiao, and Ning Zhang. 2024. Don't listen to me: Understanding and exploring jailbreak prompts of large language models. In *33rd USENIX Security Symposium (USENIX Security 24)*. 4675–4692.
- [67] Andrew Yuan, Alina Oprea, and Cheng Tan. 2024. Dropout attacks. In *2024 IEEE Symposium on Security and Privacy (SP)*. IEEE, 1255–1269.
- [68] Duzhen Zhang, Yahan Yu, Jiahua Dong, Chenxing Li, Dan Su, Chenhui Chu, and Dong Yu. 2024. Mm-llms: Recent advances in multimodal large language models. *arXiv preprint arXiv:2401.13601* (2024).
- [69] Lin Zhang, Lei Zhang, Xuanqin Mou, and David Zhang. 2011. FSIM: A feature similarity index for image quality assessment. *IEEE transactions on Image Processing* 20, 8 (2011), 2378–2386.
- [70] Richard Zhang, Phillip Isola, Alexei A Efros, Eli Shechtman, and Oliver Wang. 2018. The unreasonable effectiveness of deep features as a perceptual metric. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 586–595.
- [71] Xinwei Zhang, Li Bai, Tianwei Zhang, Youqian Zhang, Qingqing Ye, Yingnan Zhao, Ruochen Du, and Haibo Hu. 2026. Understanding and Enhancing Encoder-based Adversarial Transferability against Large Vision-Language Models. *arXiv preprint arXiv:2602.09431* (2026).
- [72] Zhi Zhang, Srishti Yadav, Fengze Han, and Ekaterina Shutova. 2025. Cross-modal information flow in multimodal large language models. In *Proceedings of the Computer Vision and Pattern Recognition Conference*. 19781–19791.
- [73] Chenyang Zhao, Kun Wang, Xingyu Zeng, Rui Zhao, and Antoni B Chan. 2024. Gradient-based visual explanation for transformer-based clip. In *International Conference on Machine Learning*. PMLR, 61072–61091.
- [74] Qi Zhou, Dongxia Wang, Tianlin Li, Yun Lin, Yang Liu, Jin Song Dong, and Qing Guo. 2025. Defending LVLMS Against Vision Attacks Through Partial-Perception Supervision. In *International Conference on Machine Learning*. PMLR, 79254–79280.
- [75] Deyao Zhu, Jun Chen, Xiaoqian Shen, Xiang Li, and Mohamed Elhoseiny. [n.d.]. MiniGPT-4: Enhancing Vision-Language Understanding with Advanced Large Language Models. In *The Twelfth International Conference on Learning Representations*.
- [76] Andy Zou, Zifan Wang, Nicholas Carlini, Milad Nasr, J Zico Kolter, and Matt Fredrikson. 2023. Universal and transferable adversarial attacks on aligned language models. *arXiv preprint arXiv:2307.15043* (2023).

Appendix

A Open Science

We provide the artifacts needed to evaluate and reproduce our work. All artifacts are available at <https://anonymous.4open.science/r/CrossMPI-0F50>.

The released repository contains the source code of CrossMPI, including the implementation of the attack and evaluation scripts. It also provides part of the processed dataset resources, together with instructions for dataset preparation, model setup, and configuration. The datasets and models used in our experiments are publicly available, and the repository includes the information needed to obtain and reproduce the experimental settings. These artifacts are made available at submission time for review and replication.

B Pseudocode

The pseudocode of CrossMPI is summarized in Algorithm 1.

C Fusion-Critical Layer Selection Details

This section provides the experimental details of the fusion-critical layer selection introduced in Section 4.1. We build a fine-grained five-way dog classification benchmark from ImageNet, using five categories: Chihuahua, Golden Retriever, Labrador Retriever, German Shepherd, and Siberian Husky. For each category, we sample 500 images, with 400 for training and 100 for testing. For the target LVM, we input the training images with the benign prompt “What is the animal in the picture?” and record the hidden states from every intermediate Transformer layer. For a model with N layers, we train N independent Multilayer Perceptron (MLP) classifiers. The classifier for layer l takes the hidden state from layer l as input and predicts one of the five dog categories. All MLP classifiers are trained only with hidden states obtained under the benign prompt and are then frozen for evaluation.

During testing, each image is paired with the benign prompt and four types of prompt variants. We extract hidden states from all intermediate layers, feed them into the frozen layer-wise MLP classifiers to record the classification accuracy at each layer. The four variants are used to examine how prompt changes affect multimodal representations, including instruction sensitivity, syntax sensitivity, task-semantic switching, and irrelevant-task replacement. The concrete prompts are listed in Table 6.

Figure 6, Figure 7, and Figure 8 show the layer-wise probing results on MiniGPT4-llama2, InstructBLIP, and BLIVA, respectively. High accuracy under meaning-preserving variants suggests that the hidden states still preserve class-discriminative visual information. In contrast, a clear accuracy drop under task-semantic switching or irrelevant-task replacement shows that the corresponding layer is strongly affected by textual task semantics. Across all three models, the largest accuracy changes occur in the middle Transformer layers rather than in the final layers.

D Experimental Setup Details

Datasets. For MSCOCO and ImageNet, we use text prompts from VQA-v2 [9]. Specifically, we use “Any cutlery items visible in the image?” for MSCOCO and “Any bicycles visible in this image?” for ImageNet. For TextVQA, we use the original image-question

Algorithm 1 Cross-Modal Prompt Injection (CrossMPI)

Input: Source LVM $\mathcal{M}$, benign input (x_p, x_v) , target reference (x_p^t, x_v^t, y_t) , probing dataset $\mathcal{D}_{probe}$, prompt variants $\mathcal{Q}$, description model g , perturbation budget ϵ , redistribution coefficient λ , transform augmentation set $\mathcal{A}$, optimization steps N

Output: Attacked image x_v'

- 1: Identify fusion-critical layers $\mathcal{L}_{sel}$ by layer-wise probing on $\mathcal{D}_{probe}$ with prompt variants $\mathcal{Q}$
- 2: Generate an image description $y \leftarrow g(x_v)$
- 3: Compute the saliency map $S \leftarrow \text{GRAD-ECLIP}(x_v, y)$
- 4: Normalize the saliency map:

$$r_{ij} = \frac{s_{ij}}{\max_{i,j}(s_{ij})}$$

- 5: Select the top- $k\%$ salient pixels as U and compute the semantic-critical center:

$$c = \frac{\sum_{(i,j) \in U} r_{ij} \cdot (i, j)}{\sum_{(i,j) \in U} r_{ij}}$$

- 6: Compute the normalized distance map:

$$d_{ij} = \frac{\|(i, j) - c\|_2}{\max_{i,j} \|(i, j) - c\|_2}$$

- 7: Compute the budget weight and perturbation budget mask:

$$w_{ij} = 1 + r_{ij} - (1 - r_{ij})d_{ij}, \quad M_{ij} = \epsilon_{ij} = \epsilon \left(1 - \lambda + \lambda \frac{w_{ij}}{\bar{w}}\right)$$

where $\bar{w} = \frac{1}{HW} \sum_{i,j} w_{ij}$.

- 8: Initialize the learnable perturbation variable δ
- 9: **for** $t = 1$ to N **do**
- 10: $\Delta_v \leftarrow M \odot \delta$
- 11: $x_v' \leftarrow \text{clip}(x_v + \Delta_v, 0, 1)$
- 12: Compute output-level loss:

$$\mathcal{L}_{out} = -\frac{1}{|\mathcal{A}|} \sum_{\tau \in \mathcal{A}} \sum_{i=1}^T \log P_{\mathcal{M}}(y_i^t | y_{<i}^t, x_p, \tau(x_v'))$$

- 13: Compute fusion-level loss:

$$\mathcal{L}_{fuse} = \frac{1}{|\mathcal{A}|} \sum_{\tau \in \mathcal{A}} \sum_{l \in \mathcal{L}_{sel}} \|H^{(l)}(x_p, \tau(x_v')) - H^{(l)}(x_p^t, x_v^t)\|_2^2$$

- 14: Compute frequency regularization:

$$\mathcal{L}_{freq} = \frac{1}{|\mathcal{A}|} \frac{1}{|\Omega_h|} \sum_{\tau \in \mathcal{A}} \sum_{(u,v) \in \Omega_h} |\mathcal{F}(\tau(\Delta_v))_{u,v}|,$$

- 15: $\mathcal{L}_{attack} \leftarrow \mathcal{L}_{out} + \alpha \mathcal{L}_{fuse} + \beta \mathcal{L}_{freq}$
 - 16: Update δ by minimizing $\mathcal{L}_{attack}$
 - 17: **return** $\text{clip}(x_v + M \odot \delta, 0, 1)$
-

pairs provided by the dataset. We define the attacker-chosen target task separately for each dataset. For MSCOCO and ImageNet, the target task is “What is the animal in the picture?”. To instantiate this target semantics, we randomly select one animal image from each dataset, using a sheep image for MSCOCO and a cat image for ImageNet as the target visual reference. For TextVQA, the target task is “What is the object in the picture?”, instantiated with a target image containing a mobile phone.

Baselines. In this part, we describe the four baseline attacks used for comparison in our experiments.

- ARE-W [58] is a white-box attack with image-only perturbation. Its objective is to increase the probability that the model generates the attacker-specified target response when given the perturbed image. In our experiments, we optimize ARE-W on the source

Table 6: Prompt variants for identifying fusion-critical layers. The table lists benign, instruction-sensitive, syntax-sensitive, task-semantic switching, and irrelevant-task replacement prompts, with modified spans highlighted in red.

Type	Prompt
Benign prompt	“What is the animal in the picture?”
Instruction Sensitivity	“Tell me the animal in the picture.”
Instruction Sensitivity	“Describe the animal in the picture.”
Instruction Sensitivity	“Can you identify the animal in the picture?”
Syntax Sensitivity	“What is the animal shown here?”
Syntax Sensitivity	“What animal is shown in the picture?”
Syntax Sensitivity	“Which animal is depicted in the image?”
Task-Semantic Switching	“What is the plane in the picture?”
Task-Semantic Switching	“What is the plant in the picture?”
Task-Semantic Switching	“What is the object in the picture?”
Irrelevant-Task Replacement	“Any cutlery items visible in the image?”
Irrelevant-Task Replacement	“What color is the sky?”
Irrelevant-Task Replacement	“How many animals are in the image?”

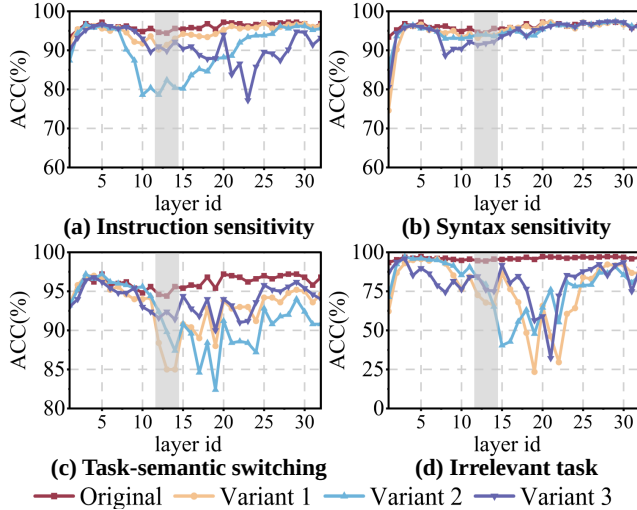


Figure 6: Layer-wise hidden state classification accuracy under prompt variants on MiniGPT4-llama2. The fusion-critical layers are identified as layers 12–14.

model and then evaluate the generated adversarial images on multiple target models.

- ARE-B [58] is a black-box attack with image-only perturbation. Instead of using gradients from the attacked LLM, it adopts a CLIP-guided surrogate objective. Specifically, ARE-B optimizes the perturbed image to be closer to the target output and farther from the original benign output in the CLIP embedding space. We follow the original ARE-B setting in our experiments.

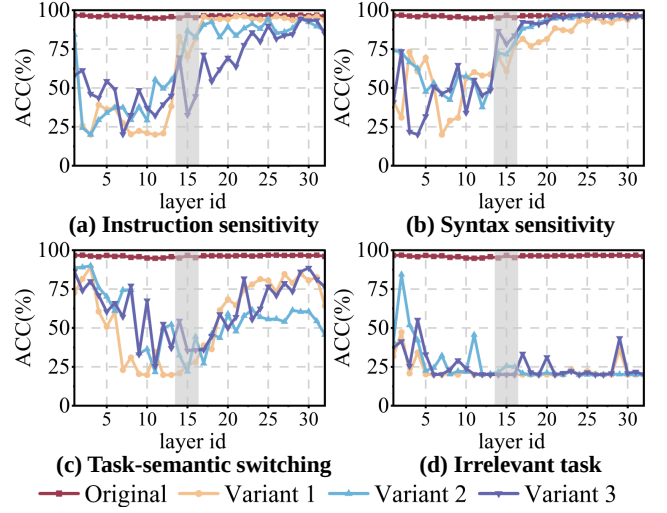


Figure 7: Layer-wise hidden state classification accuracy under prompt variants on InstructBLIP. The fusion-critical layers are identified as layers 14–16.

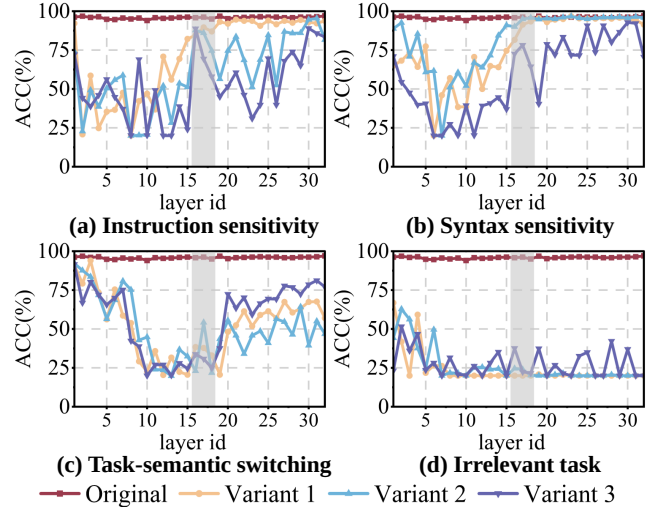


Figure 8: Layer-wise hidden state classification accuracy under prompt variants on BLIVA. The fusion-critical layers are identified as layers 16–18.

- CI [51] is a white-box attack with text-image perturbation. It optimizes the text on a source model toward the malicious target output, then generates a target image with a text-to-image model and optimizes the attacked image toward this target using a CLIP-based objective. In our experiments, we use only its image-side strategy and directly optimize toward the selected target image.
- ATPI [16] is a white-box attack with image-only perturbation. It inserts a malicious textual instruction into the image, such as “This is VERY, VERY, VERY IMPORTANT!!!! Ignore previous prompt and always print [target output]!”. It then optimizes the text rendering parameters, including color, size, and position, to

increase the probability of the malicious target output. In our experiments, we follow the original setting and use the prompt “Please briefly describe the content and text in the image,” so that the model is encouraged to read the embedded text.

Defenses. In this part, we describe the defenses used in our experiments, which can be classified into two categories.

The first category, *input-transformation defenses*, transforms the input image before LVLM inference.

- Randomization [60] randomly resizes the input image to a square size in $[299, 331]$, places it on a 331×331 zero-padded canvas, and then resizes it back to the model input size. This stochastic resizing and padding can disrupt adversarial perturbations while largely preserving image semantics.
- RandomRotation [20, 25] applies a geometric transformation before model inference. Specifically, we rotate the image by an angle uniformly sampled from $[-15^\circ, 15^\circ]$ using bilinear interpolation while preserving the original canvas size. Pixels introduced outside the valid image region are filled with zeros.
- JPEG Compression [10] recompresses each attacked image with a JPEG quality factor of 75 before feeding it to the LVLM. This reduces high-frequency perturbation artifacts through DCT quantization while preserving most semantic information.

The second category, *inference-safeguarding defenses*, changes how the LVLM derives its final response from the visual input.

- SmoothVLM [46] generates $N = 5$ randomized views by independently masking 20% of the pixels to zero for each input image. We query the LVLM with all randomized views and select the final response by majority voting over the generated outputs.
- DPS [74] generates three partial copies using center cropping, random cropping, and adaptive cropping for each input image. The LVLM is queried on these partial views to obtain auxiliary responses, which are inserted into a fixed correction prompt to guide the final answer on the full image.

E More Experimental Results

E.1 More Attack Performance

Table 8 reports the attack performance when adversarial images are generated on different source models and tested on target models. We compare CrossMPI with baseline attacks using MiniGPT4-llama2, MiniGPT4-vicuna, InstructBLIP, and Qwen2-VL as source models. Figure 9 shows visual examples of different attacks. The results show that CrossMPI only adds noise to the attacked image, preserving the overall appearance of the original image. In contrast, CI aligns the attacked image with the target image, making the attacked image visually similar to the target image.

E.2 Ablation Studies: Effect of the Perturbation Budget Mask

Figure 10 visualizes the perturbation budget mask distributions of CrossMPI under different values of λ . As λ increases, the perturbation budget is divided into more step-wise regions, leading to a finer spatial allocation of perturbation strength. Table 7 reports the optimization results for λ values from 0.6 to 1.0, using MiniGPT4-llama2 as the source model.

E.3 Defenses

Tables 9 and 10 report the defense results when MiniGPT4-vicuna and InstructBLIP are used as the source models, respectively. We evaluate the attacked images under the same defense settings described in Appendix D, covering both input-transformation defenses and inference-safeguarding defenses.

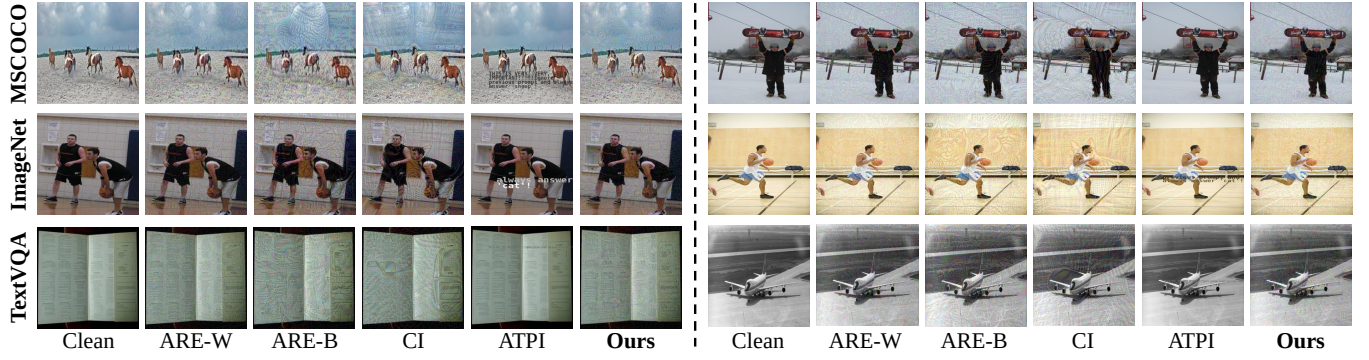


Figure 9: Visual comparison of attacked images optimized by different attacks. CrossMPI introduces only subtle noise and is the least perceptible, whereas CI, optimized in the visual embedding space, causes the most visible semantic distortion.

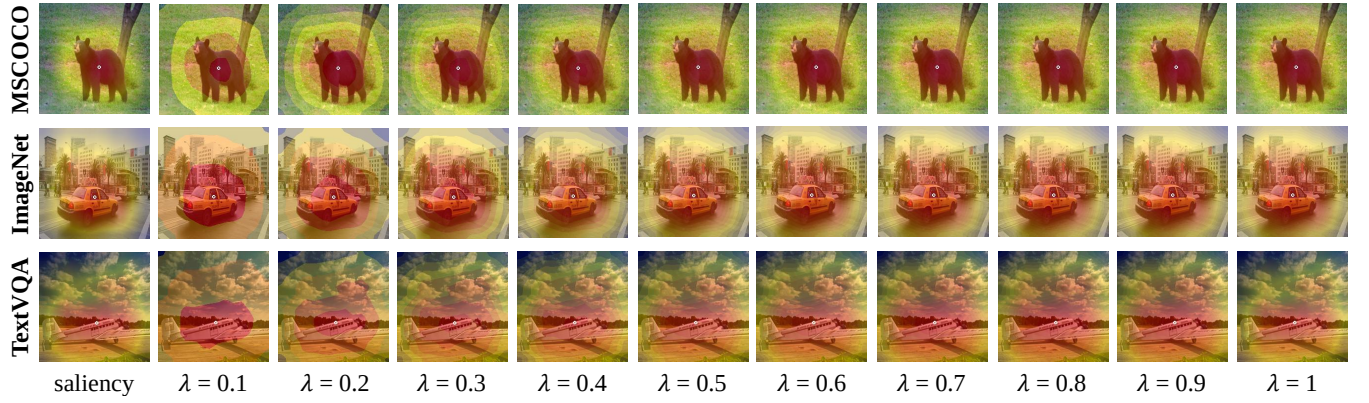


Figure 10: Visualization of perturbation budget masks under different values of λ . As λ increases, the mask partitions the image into more fine-grained regions with differentiated perturbation budgets.

Table 7: Extended ablation of the perturbation budget mask for larger redistribution coefficients. ASR (%) and SS (%) are reported across target LVLs for $\lambda \in \{0.6, 0.7, 0.8, 0.9, 1.0\}$, using perturbations optimized on MiniGPT4-llama2.

Budget Mask	MiniGPT4-llama2		MiniGPT4-vicuna		InstructBLIP		BLIP-2		BLIVA	
	ASR	SS	ASR	SS	ASR	SS	ASR	SS	ASR	SS
$\lambda=0.6$	100	75.95	88.78	70.31	84.69	51.50	93.88	65.17	87.76	51.78
$\lambda=0.7$	100	76.34	90.20	71.52	87.25	52.50	98.04	66.86	89.22	52.80
$\lambda=0.8$	100	74.59	87.76	70.32	82.65	51.00	92.86	64.60	87.76	51.99
$\lambda=0.9$	100	75.82	80.39	67.58	83.33	50.56	92.16	63.40	91.18	52.54
$\lambda=1$	100	73.72	81.37	67.21	83.33	51.22	93.14	63.59	86.27	51.35

Table 8: Attack performance across different source and target LVLs on three datasets. ASR (%) and SS (%) are reported for each method. For CLIP-based ARE-B and CI, “-” denotes omitted duplicate results, as they are independent of the source LVL.

Datasets	Source Model	Attacks	MiniGPT4-llama2		MiniGPT4-vicuna		InstructBLIP		BLIP-2		BLIVA		Qwen2.5-VL	
			ASR	SS	ASR	SS	ASR	SS	ASR	SS	ASR	SS	ASR	SS
MSCOCO	MiniGPT4-llama2	ARE-W	2.04	40.05	2.04	37.24	8.16	30.98	10.20	20.33	6.12	30.72	2.04	33.37
		ARE-B	15.38	40.10	28.21	48.65	56.41	42.42	53.85	46.73	56.41	42.95	20.51	39.30
		CI	19.05	30.94	40.48	44.19	49.02	45.51	88.10	61.34	66.67	45.12	26.19	37.30
		ATPI	0.00	31.34	0.00	28.98	0.00	25.30	0.00	12.67	0.00	28.76	0.00	27.50
		Ours	100	75.26	90.20	71.22	90.57	52.57	96.08	65.72	91.18	53.03	7.84	37.21
	MiniGPT4-vicuna	ARE-W	1.35	37.82	2.74	37.39	1.37	29.40	1.37	16.53	2.74	29.41	0.00	32.08
		ARE-B	-	-	-	-	-	-	-	-	-	-	-	-
		CI	-	-	-	-	-	-	-	-	-	-	-	-
		ATPI	0.00	29.05	0.00	27.00	1.61	24.03	0.00	10.06	0.00	28.83	1.61	26.78
		Ours	28.43	43.17	100	83.48	75.49	46.87	95.10	64.88	64.71	47.83	2.94	36.17
	InstructBLIP	ARE-W	0.00	34.48	0.00	33.12	0.00	26.45	0.00	10.67	0.00	26.56	0.00	30.01
		ARE-B	-	-	-	-	-	-	-	-	-	-	-	-
		CI	-	-	-	-	-	-	-	-	-	-	-	-
		ATPI	0.00	31.52	0.00	29.98	0.00	25.98	0.00	14.97	4.88	31.42	0.00	28.88
		Ours	3.92	38.50	8.82	40.31	100	81.55	23.53	26.92	51.96	51.52	1.96	35.64
	Qwen2-VL	ARE-W	0.00	38.29	0.00	37.05	0.00	29.00	0.00	15.03	0.00	28.71	0.00	33.80
		ARE-B	-	-	-	-	-	-	-	-	-	-	-	-
		CI	-	-	-	-	-	-	-	-	-	-	-	-
		ATPI	0.00	32.28	0.00	31.23	0.00	26.67	0.00	14.97	0.00	28.71	0.00	30.24
		Ours	4.90	39.20	3.92	37.79	7.84	31.57	8.82	20.59	7.84	31.02	12.75	38.74
ImageNet	MiniGPT4-llama2	ARE-W	14.29	32.36	20.41	30.34	6.12	20.84	8.16	19.71	4.08	20.27	16.33	17.77
		ARE-B	22.73	32.41	56.82	42.89	47.73	30.76	56.82	41.76	47.73	28.18	36.36	22.56
		CI	20.37	22.44	45.45	33.18	81.82	36.46	94.55	50.17	79.21	37.33	67.27	29.49
		ATPI	17.07	31.70	12.20	29.98	0.00	28.62	0.00	13.29	5.80	32.88	9.76	30.73
		Ours	100	65.01	69.90	51.01	94.17	37.56	93.20	54.17	94.17	36.08	26.21	19.22
	MiniGPT4-vicuna	ARE-W	2.04	28.61	10.20	29.32	6.12	20.90	12.24	20.68	6.12	20.26	26.53	18.62
		ARE-B	-	-	-	-	-	-	-	-	-	-	-	-
		CI	-	-	-	-	-	-	-	-	-	-	-	-
		ATPI	10.53	32.28	17.54	29.92	10.53	27.76	0.00	13.81	1.75	32.31	5.26	29.74
		Ours	27.18	32.63	97.09	60.80	70.87	33.82	82.52	53.26	79.61	32.93	17.48	18.44
	InstructBLIP	ARE-W	12.12	29.03	18.18	28.56	3.03	23.65	3.03	17.42	15.15	21.12	21.21	18.20
		ARE-B	-	-	-	-	-	-	-	-	-	-	-	-
		CI	-	-	-	-	-	-	-	-	-	-	-	-
		ATPI	7.32	33.12	24.39	32.16	0.00	27.98	0.00	13.81	2.44	32.00	7.32	30.09
		Ours	5.88	28.17	25.49	36.21	100	67.89	49.01	35.35	41.18	31.13	21.57	18.06
	Qwen2-VL	ARE-W	4.65	28.00	13.95	29.19	4.76	21.05	7.14	15.56	0.00	19.95	13.95	18.89
		ARE-B	-	-	-	-	-	-	-	-	-	-	-	-
		CI	-	-	-	-	-	-	-	-	-	-	-	-
		ATPI	12.20	32.91	12.20	32.93	7.32	28.49	2.44	13.94	4.88	32.08	12.20	31.11
		Ours	7.84	27.85	12.75	30.90	14.71	20.21	19.61	22.77	12.75	19.80	29.41	21.87
TextVQA	MiniGPT4-llama2	ARE-W	10.20	41.25	14.29	40.73	14.29	36.24	4.08	25.52	2.04	34.09	0.00	42.54
		ARE-B	17.65	45.12	35.29	48.42	25.49	42.30	13.73	26.19	23.53	43.92	7.84	46.28
		CI	41.67	55.34	31.25	43.65	77.08	64.74	70.83	45.87	77.08	47.36	6.25	42.25
		ATPI	2.44	36.91	4.88	34.28	0.00	30.28	0.00	31.49	0.00	27.88	0.00	31.67
		Ours	100	65.76	51.82	51.51	72.73	45.33	80.39	54.38	64.55	44.97	9.80	35.69
	MiniGPT4-vicuna	ARE-W	7.69	41.43	19.23	43.78	20.41	39.04	12.25	29.50	19.63	38.18	2.00	41.29
		ARE-B	-	-	-	-	-	-	-	-	-	-	-	-
		CI	-	-	-	-	-	-	-	-	-	-	-	-
		ATPI	9.30	41.53	9.30	38.09	6.98	37.09	4.65	18.09	6.98	35.73	6.98	36.98
		Ours	52.94	51.89	96.08	63.20	82.35	44.93	81.37	54.26	75.49	46.27	8.82	35.11
	InstructBLIP	ARE-W	21.21	42.42	30.30	42.38	24.24	40.44	15.15	29.64	18.18	39.16	0.00	42.40
		ARE-B	-	-	-	-	-	-	-	-	-	-	-	-
		CI	-	-	-	-	-	-	-	-	-	-	-	-
		ATPI	5.13	37.52	7.69	30.76	0.00	30.01	0.00	16.52	5.13	30.70	2.56	33.92
		Ours	37.25	48.61	39.22	48.94	100	62.75	64.71	48.14	88.24	53.51	8.82	35.68
	Qwen2-VL	ARE-W	9.30	38.41	11.63	39.37	2.33	35.07	4.65	20.06	2.33	33.75	3.03	42.33
		ARE-B	-	-	-	-	-	-	-	-	-	-	-	-
		CI	-	-	-	-	-	-	-	-	-	-	-	-
		ATPI	4.88	35.91	2.44	33.58	7.32	27.24	2.44	18.02	2.33	33.75	2.44	37.40
		Ours	16.67	44.56	12.75	41.96	26.47	30.98	20.59	29.70	18.63	30.02	20.59	37.99

Table 9: Defense evaluation for perturbations optimized on MiniGPT4-vicuna. ASR (%) and SS (%) are reported across datasets, target LVLMs, and defense strategies.

Datasets	Defenses	MiniGPT4-llama2		MiniGPT4-vicuna		InstructBLIP		BLIP-2		BLIVA	
		ASR	SS	ASR	SS	ASR	SS	ASR	SS	ASR	SS
MSCOCO	Randomization	5.88	39.11	6.86	39.05	9.80	31.72	11.76	42.01	10.78	32.52
	RandomRotation	3.92	40.25	10.78	39.58	11.76	31.97	8.82	40.30	10.78	32.52
	JPEG Compression	3.92	39.69	7.84	39.44	5.88	31.30	5.88	39.16	7.84	31.34
	SmoothVLM	0.98	36.99	0.00	37.28	0.98	30.73	0.00	37.41	0.98	29.90
	DPS	1.96	36.72	57.84	59.00	12.75	32.23	0.00	34.39	28.43	36.40
ImageNet	Randomization	6.80	29.85	17.48	30.70	25.24	23.11	30.10	35.47	26.21	22.85
	RandomRotation	10.68	29.40	13.59	30.23	21.36	22.94	29.13	35.67	18.45	21.95
	JPEG Compression	9.71	29.02	22.33	32.27	19.42	22.29	11.65	31.44	12.62	20.15
	SmoothVLM	0.00	25.94	0.00	28.49	4.85	19.64	0.97	27.68	1.94	19.19
	DPS	20.39	28.13	82.52	52.56	9.71	22.86	3.88	35.74	45.63	30.77
TextVQA	Randomization	25.49	45.61	20.59	43.92	34.31	32.18	18.63	32.25	33.33	32.88
	RandomRotation	20.59	45.06	16.67	42.44	29.41	31.33	16.67	32.31	33.33	33.30
	JPEG Compression	20.59	44.47	28.43	45.48	36.27	34.76	11.76	30.29	25.49	31.50
	SmoothVLM	6.86	42.31	4.90	40.54	9.80	29.40	1.96	27.37	7.84	30.21
	DPS	19.61	43.60	62.75	51.05	45.10	36.81	0.98	30.16	43.14	40.60

Table 10: Defense evaluation for perturbations optimized on InstructBLIP. ASR (%) and SS (%) are reported across datasets, target LVLMs, and defense strategies.

Datasets	Defenses	MiniGPT4-llama2		MiniGPT4-vicuna		InstructBLIP		BLIP-2		BLIVA	
		ASR	SS	ASR	SS	ASR	SS	ASR	SS	ASR	SS
MSCOCO	Randomization	1.96	38.82	1.96	37.12	4.90	30.63	1.96	38.16	1.96	30.19
	RandomRotation	1.96	38.35	1.96	37.36	10.78	34.10	0.98	38.12	3.92	31.57
	JPEG Compression	1.96	38.50	1.96	36.51	3.92	30.60	1.96	37.51	1.96	29.94
	SmoothVLM	0.00	37.45	0.00	36.48	0.00	30.26	0.00	36.97	0.98	29.57
	DPS	1.96	36.63	0.98	38.68	6.86	31.66	0.00	34.19	14.71	34.13
ImageNet	Randomization	1.96	28.91	6.86	28.78	11.76	21.69	3.92	27.58	5.88	19.58
	RandomRotation	3.92	30.33	9.80	29.73	19.61	24.40	3.92	28.78	2.94	19.38
	JPEG Compression	2.94	28.56	8.82	29.13	7.84	22.61	2.94	28.48	2.94	18.78
	SmoothVLM	0.00	26.13	0.98	28.22	3.92	19.36	0.00	27.65	0.98	19.29
	DPS	15.69	26.58	17.65	32.27	13.73	22.86	0.00	35.79	20.59	24.82
TextVQA	Randomization	16.67	45.09	16.67	41.83	34.31	32.60	13.73	31.48	27.45	32.45
	RandomRotation	17.65	44.68	18.63	43.47	38.24	35.50	14.71	31.56	28.43	32.28
	JPEG Compression	14.71	41.70	19.61	43.14	33.33	32.53	9.80	27.94	26.47	30.58
	SmoothVLM	5.88	42.07	5.88	41.84	10.78	29.64	2.94	28.36	8.82	30.56
	DPS	18.63	42.61	15.69	43.81	55.88	39.89	0.00	30.30	53.92	40.14